

République Algérienne Démocratique et Populaire
الجمهورية الجزائرية الديمقراطية الشعبية
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
وزارة التعليم العالي و البحث العلمي



المدرسة الوطنية العليا للإعلام الآلي
(المعهد الوطني للتكوين في الإعلام الآلي سابقاً)
École nationale Supérieure d'Informatique
ex. INI (Institut National de formation en Informatique)

Master Thesis

For obtaining the Master's Degree in Computer Science
Option: Information Systems and Technologies (SIT)

Scalable Architecture for Decision Support Information Systems: Towards Agile Data Modeling and Integrated Predictive Analytics

Realised by:

Mr. ABDELKEBIR Achraf

Supervised by:

Mr. ABBAS Mohamed Amir (ESI)

Promotion: **2025/2026**

Dedication

*“To my parents, for their endless love and sacrifice.
To my family and friends, for their constant support.”*

Achraf

Acknowledgements

First and foremost, I would like to express my sincere gratitude to my supervisors, Mr. ABAS Mohamed Amir and Mrs. RIHANE Yasmine, for their guidance, availability, and valuable advice throughout the development of this work. Their expertise and constructive feedback were instrumental in shaping this thesis.

I am also grateful to the staff of ESI and Ourquilane for providing a stimulating academic environment and the resources necessary to carry out this research.

Finally, I would like to extend my deepest thanks to my family and friends for their unwavering support and encouragement throughout my studies.

Abstract

Classical data warehouse architectures impose rigid structural and analytical constraints that limit the scalability of decision support information systems (DSIS): they cannot absorb schema changes and new source integrations without disruptive re-engineering, and they relegate predictive analytics to peripheral add-ons. Extending them incrementally reproduces the problem at larger scale.

This thesis frames the core challenge as a *dual-scalability deficit* — *structural scalability* (absorbing schema evolution and source heterogeneity without re-engineering) and *analytical scalability* (embedding predictive and prescriptive models natively in the decision pipeline) — and examines whether Data Vault 2.0 and the Lakehouse paradigm can be composed to close it. The central argument is that composition is achievable, but conditioned on an explicit integration contract that reconciles DV2.0’s hash-keyed relational lineage discipline with the Lakehouse’s schema-on-read file layer. Three conceptual constructs operationalize this finding: a formal dual-scalability framework (C1), a composability analysis of Data Vault 2.0 and the Lakehouse with a six-criteria evaluation instrument (C2), and a layered reference architecture with a formal integration contract evaluated against the criteria (C3).

Keywords: Data Vault 2.0, Lakehouse, Decision support system, Scalable architecture, Predictive analytics.

Résumé

Les architectures d’entrepôts de données classiques imposent des contraintes structurelles et analytiques qui limitent la scalabilité des systèmes d’information décisionnels (SID) : elles ne peuvent absorber les évolutions de schéma et l’intégration de nouvelles sources sans refonte coûteuse, et relèguent l’analytique prédictive à un rôle périphérique. Étendre ces architectures de manière incrémentale reproduit le problème à plus grande échelle.

Cette thèse cadre le défi central comme un déficit de *dual-scalabilité* : la *scalabilité structurelle* (absorption des évolutions de schéma et de l’hétérogénéité des sources sans refonte) et la *scalabilité analytique* (intégration native des modèles prédictifs et prescriptifs dans le pipeline décisionnel). Elle examine si Data Vault 2.0 et le paradigme Lakehouse peuvent être composés pour combler ce déficit. L’argument central est que la composition est réalisable, sous réserve d’un contrat d’intégration explicite qui réconcilie la discipline de lignage relationnel haché de DV2.0 avec la couche de fichiers *schema-on-read* du Lakehouse. Trois constructs conceptuelles formalisent cette réponse : un cadre formel de dual-scalabilité (C1), une analyse de composabilité de Data Vault 2.0 et du Lakehouse avec un instrument de six critères (C2), et une architecture de référence en couches avec un contrat d’intégration formel évalué selon les critères (C3).

Mots-clés : Data Vault 2.0, Lakehouse, Système d’information décisionnel, Architecture scalable, Analytique prédictive.

ملخص

تفرض معماريات مستودعات البيانات الكلاسيكية قيوداً هيكلية وتحليلية تحدّ من قابلية توسع أنظمة المعلومات القرارية: فهي عاجزة عن استيعاب تطور المخططات ودمج مصادر جديدة دون إعادة هندسة مكلفة، وتُقصي التحليلات التنبؤية إلى دور ثانوي. توسيع هذه المعماريات تدريجياً لا يحل المشكلة، بل يعيد إنتاجها على نطاق أوسع.

تُصيغ هذه الأطروحة التحدي المحوري على أنه عجز في التوسعية الثنائية: قابلية التوسع الهيكلية (استيعاب تطور المخطط وتباين المصادر دون إعادة هندسة) وقابلية التوسع التحليلية (دمج النماذج التنبؤية والتوصيفية بشكل أصيل في خط أنابيب القرار). وتفحص ما إذا كان بالإمكان تركيب Data Vault 2.0 مع نموذج Lakehouse لسد هذا العجز. الحجة المحورية هي أن التركيب ممكن، لكنه مشروط بعقد تكامل صريح يوفق بين نظام الإسناد العلانقي المُعتمد على التجزئة في DV2.0 وطبقة الملفات ذات القراءة على المخطط في Lakehouse. ثلاثة إسهامات مفاهيمية تُجسّد هذه الإجابة: إطار عمل رسمي للتوسعية الثنائية (C1)، وتحليل إمكانية التركيب بين Data Vault 2.0 و Lakehouse مع أداة تقييم سداسية المعايير (C2)، ومعمارية مرجعية متعددة الطبقات مع عقد تكامل رسمي مُقيّم وفق المعايير (C3).

الكلمات المفتاحية: Data Vault 2.0، Lakehouse، نظام معلومات قراري، معمارية قابلة للتوسع، تحليلات تنبؤية.

Contents

Dedication	i
Acknowledgements	ii
Abstract	iii
Résumé	iv
ملخص	v
List of Abbreviations	xiii
General Introduction	1
Context	1
Problem Statement	1
Research Questions	2
Objectives	2
Theoretical Constructs	3
Research Methodology	3
Thesis Outline	4
1 State of the Art and Conceptual Foundations	5
Introduction	5
1.1 Decision Support Information Systems: Concepts and Evolution	5
1.2 Classical Data Warehouse Modeling Paradigms	6
1.3 Modern Decision-Making Architectures	8
1.4 The Analytical Maturity Spectrum	10
1.5 Conceptual Framing of Scalability in DSIS	11
1.5.1 Structural Scalability	12
1.5.2 Analytical Scalability	12
1.6 Synthesis: Critical Reading and the Research Gap	14
Conclusion	15

2	Critical Analysis of Agile Modeling and Analytical Integration	16
	Introduction	16
2.1	Data Vault 2.0 and the Claim to Structural Agility	16
2.1.1	The Hub–Link–Satellite Decomposition	17
2.1.2	Why This Topology Yields Structural Agility	18
2.1.3	Operating Principles	18
2.1.4	DV2.0 Assessed Against Structural Scalability	19
2.1.5	Documented Drawbacks	20
2.2	Analytical Integration Patterns	20
2.2.1	From ETL to ELT	20
2.2.2	Native Predictive Integration	21
2.2.3	Streaming and Incremental Analytics	22
2.2.4	Analytical Integration Assessed Against Analytical Scalability	23
2.3	The Core Tension: Composing DV2.0 with the Lakehouse	23
2.3.1	The Clash, Stated Precisely	24
2.3.2	What Industry Practice Shows	24
2.3.3	Where They Align and Where They Conflict	25
2.3.4	What the Literature Does Not Resolve	26
2.4	A Criteria Grid Derived from the Analysis	27
2.4.1	Where the Criteria Come From	27
2.4.2	The Six Criteria	27
2.4.3	Assessment Rubric	28
	Conclusion	29
3	Proposed Reference Framework and Analytical Evaluation	30
	Introduction	30
3.1	Requirements and Design Rationale	31
3.2	The Reference Framework	33
3.2.1	Layered Reference Model	33
3.2.2	Component Definitions	34
3.2.3	The Integration Contract	36
3.2.4	Design Rules and Invariants	37
3.3	Evaluation Methodology	38
3.4	Comparative Study	39
3.5	Discussion	41
	Conclusion	43
	General Conclusion	44

Recall of the Problem and Approach	44
Constructs Against the Research Questions	44
Limitations	45
Future Work	45
Bibliography	47
A Annexe A — [TODO: Title]	50
B Annexe B — [TODO: Title]	51

List of Figures

1.1	Data architecture evolution: from Classical DW to Lakehouse. Each arrow labels the constraint relaxed by the transition. The Medallion pattern is an implementation discipline within the Lakehouse, not a fourth peer architecture. <i>Author-generated.</i>	9
1.2	Construct C1 as two orthogonal axes. Structural scalability (horizontal) and analytical scalability (vertical) are independent: each surveyed approach sits in a different quadrant, and only the top-right region satisfies both. Tick abbreviations match Table 1.4 (Src, Sch, Vol, His, Prp; Mat, ML, Rdy, Wkl, Lif). <i>Author-generated.</i>	13
1.3	Surveyed architectures assessed against the C1 dual-scalability dimensions. Columns are grouped by axis (Structural left, Analytical right). Cell shading encodes the ordinal score: ✓ Strong (dark) / ~ Partial (mid) / × Weak (light). Scores are author assessments, not measured values. Column abbreviations: Src source integration, Sch schema evolution, Vol volume, His historization, Prp change propagation, Mat maturity reach, ML native model integration, Rdy data readiness, Wkl workload plurality, Lif lifecycle support. <i>Author-generated.</i>	15
2.1	Data Vault 2.0 reference architecture. Source: Linstedt and Olschimke, 2015.	17
2.2	Data Vault 2.0 topology. <i>Author-generated.</i>	17
2.3	Native predictive pipeline on the Lakehouse. <i>Author-generated.</i>	22
2.4	Data Vault 2.0 mapped onto the Lakehouse medallion. Source: Databricks, 2022 (<i>industry/vendor source; not peer-reviewed</i>).	25
2.5	Provenance of the C2 criteria. The ★ marks the author’s added criterion. <i>Author-generated.</i>	27
3.1	Requirements derivation. Each research question forces a block of requirements, which in turn serve the C1 scalability dimensions. RQ3 is the only question that binds both axes — the composability requirements (R10–R12) are what couple structure to analytics. <i>Author-generated.</i>	31

3.2	The layered reference framework. Data flows bottom-up; the native analytics layer (4) reads from the agile modeling layer (2) directly, not through the serving marts (3). The open table format is the shared physical substrate; the governance plane cross-cuts every layer. <i>Author-generated</i>	34
3.3	The integration contract as a provider–consumer interface over a shared storage substrate. The modeling layer guarantees four entity properties; the analytics layer relies only on them. <i>Author-generated</i>	36
3.4	Conditions of applicability. The framework is worth its complexity in the high–high quadrant; in the other three, a simpler architecture meets the need at lower cost. <i>Author-generated</i>	42

List of Tables

1.1	Evolution of decision support paradigms.	6
1.2	Classical data warehouse modeling paradigms: comparative overview. . . .	8
1.3	Analytical maturity spectrum and corresponding architectural requirements.	10
1.4	Dual-scalability dimensions: Construct C1.	13
2.1	Data Vault 2.0 assessed against C1 structural-scalability dimensions. Scores are author assessments. <i>Author-generated.</i>	19
2.2	ETL versus ELT in the modern data pipeline. <i>Author-generated.</i>	21
2.3	The Lakehouse analytics stack assessed against C1 analytical-scalability dimensions. Scores are author assessments. <i>Author-generated.</i>	23
2.4	The clash in one view: relational-lineage discipline versus file-based flexibility. <i>Author-generated.</i>	24
2.5	Alignments and conflicts between Data Vault 2.0 and the Lakehouse, paired by theme. <i>Author-generated.</i>	26
2.6	Construct C2 — the six evaluation criteria and their C1 anchors. <i>Author-generated.</i>	28
2.7	C2 assessment rubric. For every criterion the High column is the favourable end, including implementation complexity, where High denotes low burden. <i>Author-generated.</i>	29
3.1	Requirements traceability: each requirement maps to the research question that forces it and the C1 scalability dimension it serves. Blocks are color-coded by axis — structural (blue), analytical (green), composability and governance (orange). <i>Author-generated.</i>	32
3.2	Component definitions across the five layers: inputs, outputs, responsibility, and the C1 dimension each serves. <i>Author-generated.</i>	35
3.3	The integration contract in six clauses: each provider guarantee, its realizing mechanism, and the requirement it discharges. <i>Author-generated.</i>	36
3.4	Threats to the validity of the evaluation, with mitigations. <i>Author-generated.</i>	39

3.5	Evaluation matrix: four approaches against the six C2 criteria, scored on the rubric of Table 2.7. The proposed framework leads on structure, analytics, and composability, but not on query performance or implementation complexity. <i>Author-generated.</i>	40
3.6	How Chapter 3 addresses the three research questions. <i>Author-generated.</i> .	43

List of Abbreviations

ACID	Atomicity, Consistency, Isolation, Durability
AI	Artificial Intelligence
API	Application Programming Interface
BI	Business Intelligence
C1	Construct 1 — Dual-scalability conceptualization
C2	Construct 2 — Composability analysis and criteria instrument
C3	Construct 3 — Reference framework and integration contract
CDC	Change Data Capture
CIF	Corporate Information Factory
DAG	Directed Acyclic Graph
DDL	Data Definition Language
DML	Data Manipulation Language
DSIS	Decision Support Information System
DSS	Decision Support System
DV2.0	Data Vault 2.0
DW	Data Warehouse
ELT	Extract, Load, Transform
ESI	École nationale Supérieure d’Informatique
ETL	Extract, Transform, Load
JSON	JavaScript Object Notation
KPI	Key Performance Indicator

MDM Master Data Management

ML Machine Learning

ODS Operational Data Store

OLAP Online Analytical Processing

OLTP Online Transaction Processing

RDBMS Relational Database Management System

RQ Research Question

SCD Slowly Changing Dimension

SQL Structured Query Language

General Introduction

Context

Three pressures are converging on the systems organizations use to make decisions, and together they outrun the assumptions on which those systems were built.

The first pressure is organizational agility. Business environments now change faster than fixed schemas can follow. Mergers, new product lines, and regulatory shifts reshape an organization's information needs on a timescale of months. Each such event invalidates data models that were stable the year before Kleppmann, 2017. A schema fixed at design time becomes a liability the moment the business it describes moves.

The second pressure is source heterogeneity and velocity. The number of data sources feeding the decision layer has grown sharply: transactional systems, streaming feeds, semi-structured and unstructured data, and change-data-capture pipelines all arrive at once. The volume and variety now exceed what a pre-designed schema can absorb without structural disruption Armbrust et al., 2020; Vassiliadis, 2009. Integration that once targeted a handful of relational sources must now accommodate formats that were never specified in advance.

The third pressure is rising analytical maturity. Organizations are moving along the descriptive, diagnostic, predictive, and prescriptive spectrum Davenport and Harris, 2007. Most remain at descriptive or diagnostic business intelligence, yet demand for predictive and prescriptive capability is growing steadily Chen et al., 2012. The decision layer is now expected to forecast and recommend, not only to report what already happened.

These pressures bear on a specific object of study: *Decision Support Information Systems* (DSIS) built on data-warehouse architectures. Classical warehouse designs — the Inmon Corporate Information Factory Inmon, 2005 and the Kimball dimensional model Kimball and Ross, 2013 — were engineered for a stable, pre-specified analytical perimeter. That assumption no longer holds.

Problem Statement

Classical DSIS architectures force organizations to commit prematurely and rigidly to two things at once. The first is a fixed structural schema: column names, table shapes, and entity definitions decided during design and expensive to revise once data ingestion has begun. The second is a fixed analytical scope: typically descriptive reporting, which cannot be

extended toward predictive or prescriptive capability without grafting external systems onto the pipeline.

Both commitments are made before requirements stabilize. When the structure must change, the result is costly re-engineering of tables, mappings, and downstream reports. When analytical ambition grows, predictive capability is bolted on downstream rather than designed into the architecture. The commitment is premature because the business it serves is still moving, and rigid because revising either decision is disproportionately expensive.

The literature addresses these two failures separately. Structural agility is studied through Data Vault 2.0 Linstedt and Olschimke, 2015 and schema-evolution patterns Kleppmann, 2017. Analytical integration is studied through the Lakehouse paradigm Armbrust et al., 2021 and open table formats such as Delta Lake Armbrust et al., 2020. The two run as distinct research conversations. No consolidated reference framework treats both dimensions of scalability jointly. The composability of the leading structural approach, Data Vault 2.0, with the leading open-format analytical paradigm, the Lakehouse, is asserted in industry practice but under-examined conceptually.

This thesis takes that gap as its starting point. Under an explicit integration contract, the relational-lineage discipline of Data Vault 2.0 and the schema-on-read file layer of the Lakehouse compose into a coherent scalable DSIS. The conditions under which this holds, and the trade-offs it incurs, are the subject of the chapters that follow.

Research Questions

RQ1 (structural): How can a DSIS architecture absorb new sources, schema changes, and volume growth without disruptive re-engineering?

RQ2 (analytical): How can predictive/prescriptive capability be integrated *natively* into the decision pipeline rather than bolted on downstream?

RQ3 (composability): Under what conditions can an agile relational-lineage modeling discipline (Data Vault 2.0) and a file-based Lakehouse paradigm be composed into a single coherent architecture, and what are the trade-offs?

Objectives

This thesis pursues four objectives, each tied to one or more research questions.

- Survey the landscape of DSIS architectural paradigms — classical (Inmon, Kimball) and modern (Data Vault 2.0, Lakehouse, Delta Architecture) — to identify structural and analytical capabilities and gaps. (→ RQ1, RQ2)

- Critically analyze the properties of Data Vault 2.0 and the Lakehouse paradigm along structural and analytical dimensions, identifying the conditions and tensions relevant to their composition. (→ RQ3)
- Construct a layered reference architecture that integrates Data Vault 2.0 as the agile structural layer with the Lakehouse as the native analytical layer, defined by a formal integration contract. (→ RQ1, RQ2, RQ3)
- Evaluate the proposed framework analytically against a criteria instrument derived from the critical analysis, using a non-empirical, literature-grounded comparative approach. (→ RQ3)

Theoretical Constructs

The thesis develops three theoretical constructs, one per chapter.

- **C1 — Dual-scalability conceptualization** (Chapter 1): a formal definition of structural scalability and analytical scalability as distinct but coupled dimensions of DSIS evolution, with operationalizing criteria for each.
- **C2 — Composability analysis and criteria instrument** (Chapter 2): a critical examination of whether Data Vault 2.0 and the Lakehouse paradigm compose, culminating in a six-criteria evaluation grid grounded in the literature.
- **C3 — Reference framework and integration contract** (Chapter 3): a layered reference architecture with a formally stated integration contract between the agile structural layer and the native analytics layer, evaluated against the C2 criteria grid.

Research Methodology

The thesis is theoretical and proceeds without a prototype or experiment. It uses three methods.

The first is a narrative and semi-systematic literature review. The review covers foundational texts (Inmon, Kimball, Linstedt) and peer-reviewed literature from 2020–2026, supplemented by authoritative industry and vendor primary sources that reflect current practice. Industry sources are flagged explicitly and are not treated as peer-reviewed evidence.

The second is conceptual design. The reference framework (C3) is constructed through conceptual synthesis: defining layers, components, interfaces, and invariants, without a software prototype.

The third is analytical, non-empirical evaluation. The framework is evaluated against the C2 criteria instrument through structured comparison with existing documented implementations. Validation is conceptual rather than experimental, and this thesis states that limit openly rather than concealing it.

Thesis Outline

Chapter 1 surveys the state of the art in DSIS architectures and introduces the dual-scalability conceptualization (C1). Chapter 2 critically analyzes Data Vault 2.0 and the Lakehouse paradigm, develops the composability analysis, and delivers the six-criteria evaluation instrument (C2). Chapter 3 constructs the layered reference framework with its integration contract and conducts the analytical evaluation (C3). The General Conclusion answers RQ1, RQ2, and RQ3 explicitly, states limitations, and identifies directions for future work.

Chapter 1

State of the Art and Conceptual Foundations

Introduction

This chapter builds the conceptual instrument the rest of the thesis depends on. The survey literature on decision support architectures is large, but it treats structural evolution and analytical capability as separate concerns. No single lens reads an architecture for both at once. This chapter supplies that lens.

The argument moves from vocabulary to instrument:

- §1.1 positions the data warehouse as the operational core of DSIS and traces the move from reporting to modern analytical platforms.
- §1.2 examines the two classical modeling paradigms and the shared assumption that limits them.
- §1.3 surveys the modern architectures that arose to relax that assumption.
- §1.4 sets out the analytical maturity spectrum and the demand each level places on the architecture.
- §1.5 consolidates these observations into the dual-scalability conceptualization, the construct C1.
- §1.6 re-reads every surveyed approach through that lens and isolates the research gap.

The survey sections are not neutral. Each one asks a single question about every paradigm it describes: *does this property concern structural evolution or analytical capability?* That question is what turns a reading list into the instrument delivered in §1.5.

1.1 Decision Support Information Systems: Concepts and Evolution

Decision Support Information Systems (DSIS) are computer-based systems that supply managers with the data and analysis needed for non-routine decisions. Four terms recur

throughout the thesis and are fixed here:

- **DSS (Decision Support System)** the origin concept: a system that supports, rather than automates, managerial decisions.
- **BI (Business Intelligence)** the technologies and practices for gathering, integrating, and analyzing information to improve decision making Watson, 2009.
- **Data warehouse (DW)** the integrated, query-oriented store that feeds the analytical layer; the operational core of the DSIS.
- **DSIS (as used here)** the data-warehouse-centric decision support systems that descend from the DSS concept and its consolidation under BI.

The field evolved through four eras, each widening the analytical scope of the decision layer while preserving the data warehouse at its core. Table 1.1 summarizes the progression.

Era	Dominant paradigm	Analytical scope
1960s–1980s	Management information systems	Reporting
1990s–2000s	Data warehouse + OLAP	Descriptive
2000s–2010s	Business intelligence platforms	Descriptive + diagnostic
2010s–present	Modern platforms (Lakehouse, cloud DW)	Full spectrum

Table 1.1 — Evolution of decision support paradigms.

The table makes one constant visible: every era kept the data warehouse as its core while widening the questions the system could answer Chen et al., 2012; Davenport and Harris, 2007. A peer-reviewed survey traces the same arc at the storage layer, from data warehouse to data lake to Lakehouse Nambiar and Mundra, 2022. Across all four eras runs a single assumption — that requirements can be specified before ingestion begins. That assumption is exactly what classical warehouse modeling encodes, and it is where the next section opens.

1.2 Classical Data Warehouse Modeling Paradigms

Two modeling paradigms have governed data warehouse design for three decades. They differ sharply in approach yet share the limitation that motivates this thesis: both assume a stable, pre-specified schema. A third paradigm, Data Vault 2.0, was built to escape that limitation and is previewed at the end of this section.

Inmon — Corporate Information Factory (CIF). A normalized, enterprise-wide warehouse built top-down Inmon, 2005.

- **Core** a subject-oriented, integrated, non-volatile, time-variant repository in third normal form (3NF), from which dependent data marts are derived.
- **Strength** enterprise integration — one normalized model enforces consistency across the organization.
- **Weakness** long time-to-value and schema rigidity — the enterprise model must be largely complete before it delivers, and structural revision propagates through the normalized core.

Kimball — dimensional model. A warehouse built bottom-up from business processes Kimball and Ross, 2013.

- **Core** star and snowflake schemas of fact tables surrounded by conformed dimensions; slowly changing dimension (SCD) types 1, 2, and 3 record attribute history at different fidelities; the ETL discipline is codified separately Kimball and Caserta, 2004.
- **Strength** query simplicity and fast delivery of business intelligence.
- **Weakness** schema changes are expensive and auditability is partial — SCD mechanisms record only the changes the designer anticipated.

The shared constraint. Read against a formal design perspective, the two paradigms diverge on method but converge on a constraint Golfarelli and Rizzi, 2009. Both assume a stable analytical perimeter and a schema fixed at design time. Neither was engineered for continuous schema evolution, and neither offers a native path toward predictive or prescriptive capability. The rigidity is built into the modeling discipline itself, not into any particular implementation.

Data Vault 2.0 (preview). The third major paradigm, designed specifically to absorb schema evolution and source change without re-engineering Linstedt and Olschimke, 2015. It decomposes the model into *hubs*, *links*, and *satellites* so that structural change is additive rather than destructive. This thesis previews it here only to complete the comparative frame; Chapter 2 examines its mechanics and limits in depth.

Table 1.2 positions all three paradigms across the properties the dual-scalability lens later formalizes.

Property	Inmon CIF	Kimball	Data Vault 2.0
Modeling approach	Top-down, enterprise	Bottom-up, process	Source-aligned, additive
Schema type	Normalized (3NF)	Dimensional (star)	Hub–link–satellite
Integration strategy	Central normalized core	Conformed dimensions	Business keys + links
Time-to-value	Long	Fast	Moderate
Schema flexibility	Low	Low	High
Auditability	Partial	Partial (SCD)	Full, non-destructive
Analytical scope	Descriptive	Descriptive	Descriptive

Table 1.2 — Classical data warehouse modeling paradigms: comparative overview.

The three paradigms differ widely on structural properties yet remain identical on analytical scope — all capped at descriptive reporting. The modern architectures surveyed next emerged precisely to relax the limits this table exposes.

1.3 Modern Decision-Making Architectures

Modern architectures arose to absorb pressures the classical warehouse could not: data velocity, source heterogeneity, and demand for machine learning on the same data. Each relaxes a specific constraint. Read through the dual-scalability lens, they target the analytical and volume dimensions far more than the structural-modeling one.

Lambda architecture. Addresses the trade-off between latency and accuracy with two parallel pipelines Marz and Warren, 2015 (*industry practice; not peer-reviewed*).

- **Batch layer** recomputes accurate views over an immutable master dataset.
- **Speed layer** serves low-latency approximate results over recent data.
- **Serving layer** merges the two.
- **Cost** the same business logic is implemented and maintained twice, once per layer.

Kappa architecture. Eliminates that duplication by treating all processing as stream processing over a replayable log Kreps, 2014 (*industry practice; not peer-reviewed*). A single code path handles real-time and historical computation; history is reprocessed by replaying the log. Lambda and Kappa both relieve velocity pressure, but neither introduces a discipline for structural schema evolution — both assume the *processing logic*, not the source schema, is what changes.

Data lake → Lakehouse. A shift toward storage flexibility and workload plurality.

- **Data lake** introduced schema-on-read over open file formats, accepting raw data of any shape — but lacked governance, transactional semantics, and BI-grade query performance.
- **Lakehouse** layers ACID table semantics and a metadata layer over open columnar formats such as Parquet and ORC Armbrust et al., 2021. Open table layers — Delta Lake Armbrust et al., 2020, Iceberg, Hudi — add transactions, time travel, and schema enforcement directly on the lake, so BI and ML run on one governed store. Peer-reviewed surveys confirm this as the consolidating direction of enterprise data architecture Harby and Zulkernine, 2024; Schneider et al., 2024.
- **Gap** the open table layer governs *file* transactions and schema enforcement, not entity-level lineage or historization discipline — the structural-modeling gap remains.

Delta architecture (medallion). The dominant implementation pattern for the Lakehouse Databricks, 2022 (*industry practice; not peer-reviewed*), organized as three zones:

- **Bronze** raw ingested data.
- **Silver** cleansed and conformed data.
- **Gold** business-ready aggregates.

The pattern maps the classical ETL/ELT pipeline Vassiliadis, 2009 onto governed table layers, so refinement is incremental and each zone is independently queryable.

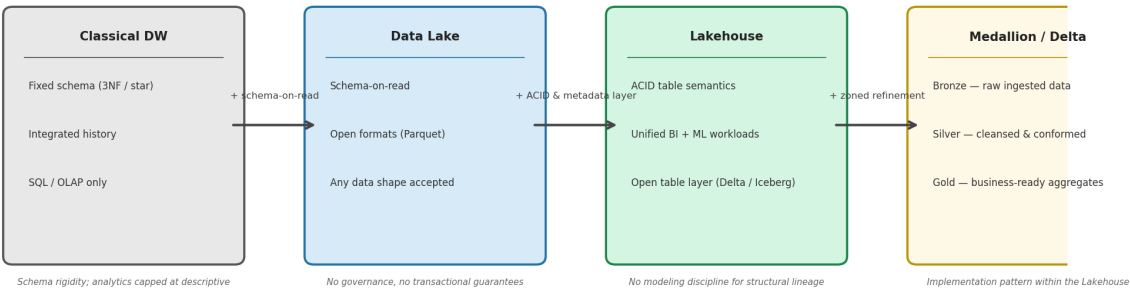


Figure 1.1 — Data architecture evolution: from Classical DW to Lakehouse. Each arrow labels the constraint relaxed by the transition. The Medallion pattern is an implementation discipline within the Lakehouse, not a fourth peer architecture. *Author-generated.*

These architectures advance the analytical and volume frontiers, yet none supplies the modeling discipline that governs structural lineage and historization. Their value, and their gap,

becomes precise only once the analytical demand they serve is specified — which the maturity spectrum does next.

1.4 The Analytical Maturity Spectrum

Analytical capability progresses through four levels, and each level imposes a sharper architectural demand than the last Gartner, 2012 (*industry practice; not peer-reviewed*):

- **Descriptive** — *what happened?*
- **Diagnostic** — *why did it happen?*
- **Predictive** — *what will happen?*
- **Prescriptive** — *what should be done?*

The requirements rise steeply across these steps. Descriptive and diagnostic analysis are served by the data warehouse and OLAP. Predictive analysis requires feature pipelines and a machine learning training capability grounded in statistical learning Jordan and Mitchell, 2015. Prescriptive analysis requires a closed feedback loop, model governance, and data fresh enough to act on; where decisions run over streaming data it also requires online learning over evolving distributions Bifet et al., 2018. The higher levels are not extensions of reporting — they demand a different pipeline shape. Table 1.3 maps each level to its requirement.

Level	Question answered	Key capability required	Typical architecture
Descriptive	What happened?	Aggregation, reporting	DW + OLAP
Diagnostic	Why did it happen?	Drill-down, correlation	DW + OLAP
Predictive	What will happen?	Feature pipelines, ML training	ML platform + features
Prescriptive	What should be done?	Feedback loops, model governance, fresh data	Closed-loop pipeline

Table 1.3 — Analytical maturity spectrum and corresponding architectural requirements.

Figure 1.4 renders this as a rising staircase: the jump from diagnostic to predictive is where the architecture, not just the query, must change. Yet the empirical reality is that most organizations remain at the descriptive and diagnostic levels: industry surveys report that the large majority have not reached transformational analytics maturity Gartner, 2018 (*industry practice; not peer-reviewed*). A peer-reviewed review of maturity models adds the relevant caution — the consulting models in common use lack theoretical grounding, leaving maturity defined without a rigorous instrument “Understanding Data and Analytics Maturity: A Systematic Review of Maturity Model Composition,” 2024. That absence is part of what the next section repairs.

The implication is structural, not incremental. Reaching the prescriptive level requires the decision pipeline to be designed for native analytical capability from the outset, not extended with downstream additions. This is the demand that the *analytical scalability* axis — one half of the dual construct defined next — is built to measure.

1.5 Conceptual Framing of Scalability in DSIS

The survey reveals a consistent split: existing approaches advance either structural evolution or analytical integration, but never frame both as dimensions of one property, and never combine them into a single instrument. Data Vault 2.0 advances structural evolution; the Lakehouse advances analytical integration. Neither is articulated as a form of *scalability*, and the two are never read together. This section closes that gap by formalizing both as the construct **C1**.

Definition (C1). *Scalability in DSIS* is the capacity of the system to grow — structurally and analytically — without re-engineering the existing architecture.

The definition resolves into two distinct axes, each made assessable by five concrete dimensions:

- **Structural scalability** how the system absorbs change in its data and sources.
- **Analytical scalability** how far the system can progress in analytical capability.

1.5.1 Structural Scalability

The capacity of a DSIS to evolve its data structure and integration footprint without disruptive re-engineering. Without it, every new source or schema change forces a partial rebuild. Five dimensions characterize it:

- **Source integration** the cost of absorbing a new heterogeneous data source, measured in effort, schema impact, and pipeline disruption.
- **Schema evolution** the cost and blast radius of structural changes (column renames, entity splits, type changes), and the degree to which they are non-destructive and version-aware.
- **Volume and throughput growth** the capacity to scale data volume and ingestion load without architectural redesign.
- **Historization and auditability** the completeness and non-destructiveness of the historical record, and the ability to audit changes over time.
- **Loosely-coupled change propagation** the degree to which a structural change in one source or layer is isolated from downstream consumers.

1.5.2 Analytical Scalability

The capacity of a DSIS to progress along the maturity spectrum — descriptive toward prescriptive — with advanced capability integrated *natively* in the decision pipeline rather than bolted on downstream. Without it, predictive and prescriptive capability can only be grafted externally, outside the governed pipeline. Five dimensions characterize it:

- **Maturity reach** the highest maturity level the architecture supports natively, from descriptive through prescriptive.
- **Native model integration** the degree to which ML and predictive models are first-class citizens of the pipeline rather than external appendages.
- **Data readiness for analytics** the freshness, trust, and feature availability of data at the point of analytical consumption.
- **Workload plurality** the capacity to serve BI and ML workloads from the same governed data layer without duplication.
- **Lifecycle support** the support for model retraining, feedback loops, and model governance within the pipeline.

Axis	Dimension	Definition
	Source integration	Cost of absorbing a new heterogeneous source
	Schema evolution	Cost and isolation of structural changes
	Volume / throughput	Scaling capacity without redesign
	Historization	Non-destructive full-history retention
	Change propagation	Blast-radius isolation of structural change
	Maturity reach	Highest maturity level natively supported
	Native model integration	ML as first-class pipeline citizen
	Data readiness	Freshness, trust, feature availability
	Workload plurality	BI + ML on same governed layer
	Lifecycle support	Retraining, feedback, model governance

Table 1.4 — Dual-scalability dimensions: Construct C1.

Table 1.4 presents the ten dimensions constituting C1; Chapters 2 and 3 evaluate existing approaches and the proposed framework against them.

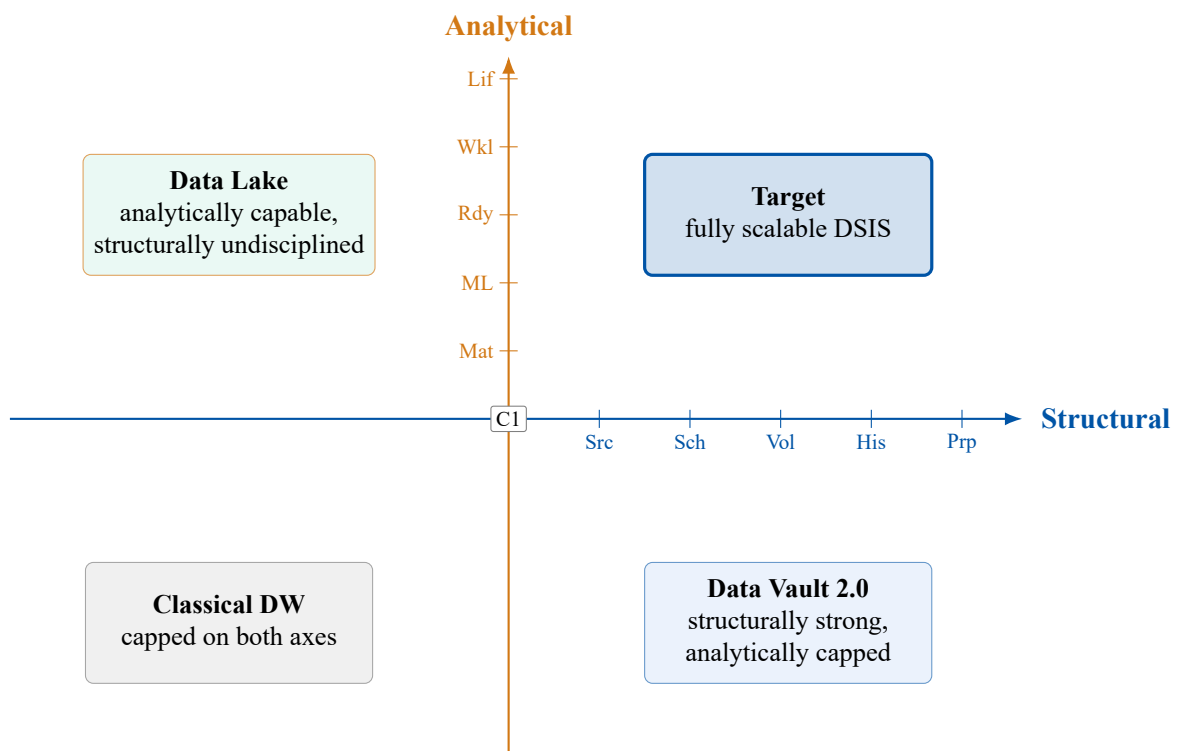


Figure 1.2 — Construct C1 as two orthogonal axes. Structural scalability (horizontal) and analytical scalability (vertical) are independent: each surveyed approach sits in a different quadrant, and only the top-right region satisfies both. Tick abbreviations match Table 1.4 (*Src*, *Sch*, *Vol*, *His*, *Prp*; *Mat*, *ML*, *Rdy*, *Wkl*, *Lif*). *Author-generated*.

The two axes are orthogonal (Figure 1.2). A system can be:

- **Structurally scalable yet analytically capped** — as classical Data Vault 2.0 is, absorbing schema change yet reaching no further than descriptive analysis.
- **Analytically capable yet structurally undisciplined** — as a raw data lake is, supporting ML yet governing neither lineage nor historization.

A coherent scalable DSIS must therefore address both axes jointly, not trade one for the other. The next section applies this lens to every surveyed approach and shows that none does.

1.6 Synthesis: Critical Reading and the Research Gap

Read through the dual-scalability lens, no surveyed approach scores well on both axes at once. Each is strong on a subset of dimensions and weak on the rest, and that pattern of strengths and weaknesses *is* the research gap. The four candidate approaches assess as follows:

Classical paradigms (Inmon, Kimball). Moderate structurally, capped analytically. Volume growth is manageable, but schema evolution is poor, source integration is rigid, historization is limited to anticipated SCD types, and change propagation is tightly coupled through the normalized core or conformed dimensions. Analytically they stop at descriptive and diagnostic and cannot progress without an external bolt-on.

Data lake. Inverts that profile on the structural side. It scales volume and absorbs heterogeneous sources easily, but imposes no modeling discipline — schema-evolution governance, historization, and change-propagation isolation are all weak. Its analytical ceiling is potentially high, yet poor data quality and absent governance undermine data readiness, so the potential is rarely realized.

Lakehouse. Strong analytically, partial structurally. ACID semantics, unified BI and ML workloads, and open formats give it strength across every analytical dimension. Structurally it handles volume and schema-on-read well, but it provides no modeling discipline for structural lineage and historization equivalent to Data Vault 2.0.

Data Vault 2.0. Strong structurally, limited analytically — the mirror image of the Lakehouse. It is designed for schema evolution, source integration, historization, and change propagation, but does not natively support ML workloads or progression along the maturity spectrum. Chapter 2 examines this profile in depth.

Figure 1.3 consolidates the four assessments against all ten C1 dimensions.

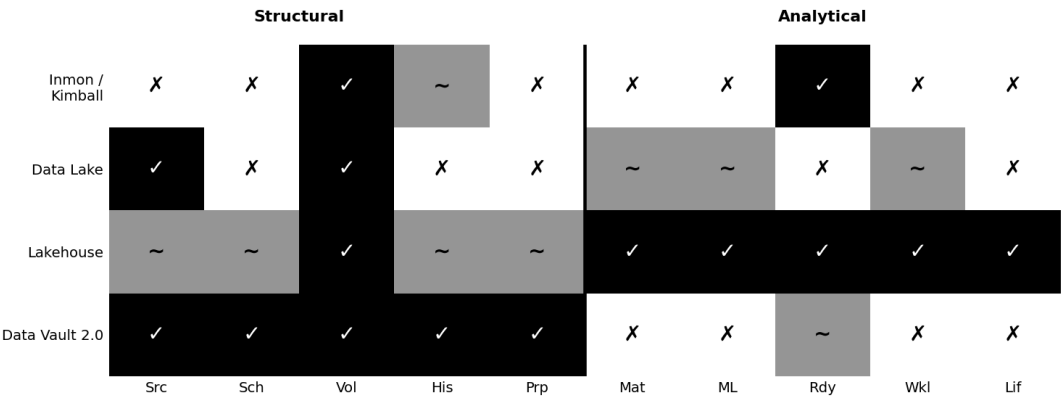


Figure 1.3 — Surveyed architectures assessed against the C1 dual-scalability dimensions. Columns are grouped by axis (Structural left, Analytical right). Cell shading encodes the ordinal score: ✓ Strong (dark) / ~ Partial (mid) / × Weak (light). Scores are author assessments, not measured values. Column abbreviations: Src source integration, Sch schema evolution, Vol volume, His historization, Prp change propagation, Mat maturity reach, ML native model integration, Rdy data readiness, Wkl workload plurality, Lif lifecycle support. *Author-generated.*

Figure 1.3 makes the gap geometric: strength clusters on one half or the other, and no row fills both halves. The structural strengths of Data Vault 2.0 and the analytical strengths of the Lakehouse are near-complementary — their fills are close to jigsaw pieces. Their combination is therefore the most promising candidate for a jointly scalable DSIS, yet that composability is asserted in industry practice and under-examined conceptually. That examination is the task of Chapter 2.

Conclusion

This chapter delivered two things:

- **A grounded survey** — the vocabulary of DSIS, from the classical warehouse paradigms through the modern Lakehouse and the analytical maturity spectrum.
- **The construct C1** (§1.5) — structural and analytical scalability as two orthogonal axes, each operationalized by five assessable dimensions: the single lens the literature had been missing.

Applying C1 in §1.6 exposed the gap. No single existing approach is strong on both axes, and the near-complementary profiles of Data Vault 2.0 and the Lakehouse make their combination the leading candidate — with their composability still unexamined. Chapter 2 subjects Data Vault 2.0 and the Lakehouse to a systematic critical analysis along these dimensions, developing the composability examination (C2).

Chapter 2

Critical Analysis of Agile Modeling and Analytical Integration

Introduction

Chapter 1 established that scalability in decision support architectures has two independent dimensions: structural scalability, the capacity to absorb source change without re-engineering; and analytical scalability, the capacity to reach progressively higher levels of the decision support maturity spectrum. It also established that no surveyed paradigm satisfies both simultaneously. That finding is the entry point for this chapter.

This chapter moves from survey to critical analysis. It examines the two candidate components of a dual-scalable architecture — Data Vault 2.0 on the modeling side, and the Lakehouse’s native analytics stack on the analytical side — not as independent topics but as elements that must eventually compose. The four sections converge toward a single question: do they actually compose, and if so, under what conditions and with what trade-offs?

2.1 Data Vault 2.0 and the Claim to Structural Agility

Data Vault 2.0 is the modeling discipline this thesis nominates for the structural axis of a dual-scalable DSIS Linstedt and Olschimke, 2015. Its central claim is that structural change can be made additive rather than destructive. This section examines the mechanics behind that claim, scores them against the C1 structural dimensions defined in Section 1.5, and states the drawbacks honestly, because those drawbacks set up the composability tension developed later in the chapter.

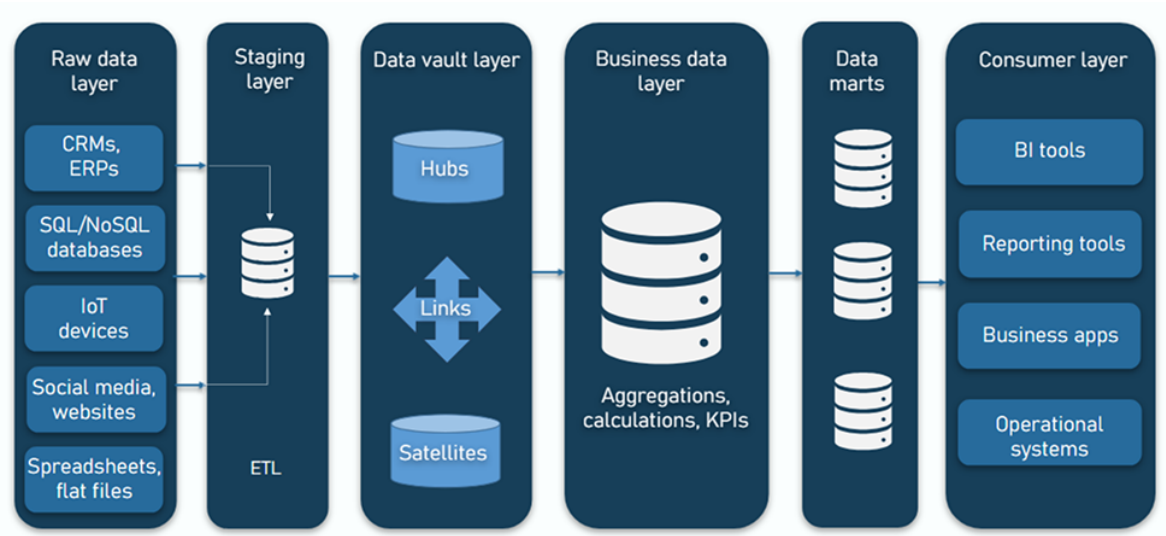


Figure 2.1 — Data Vault 2.0 reference architecture. Source: Linstedt and Olschimke, 2015.

Figure 2.1 situates the modeling layer within the broader Data Vault 2.0 architecture, from staging through the raw and business vaults to the information delivery layer.

2.1.1 The Hub–Link–Satellite Decomposition

Data Vault 2.0 organizes every entity across three distinct object types, each holding a different kind of information Linstedt and Olschimke, 2015.

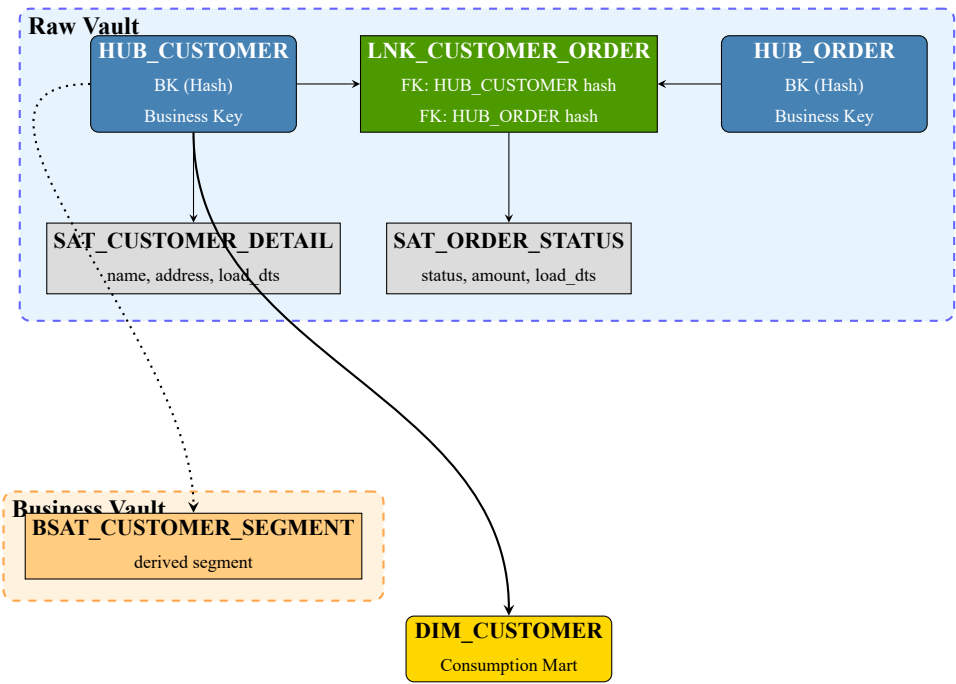


Figure 2.2 — Data Vault 2.0 topology. Author-generated.

- **Hub** — Stores only the business key: the stable, source-independent identifier the

organization uses to recognize an entity (a customer, an order, a product). A Hub holds no attributes and no relationships. Once created, it never changes; absorbing a new source entity means adding a new Hub alongside the existing ones.

- **Link** — Stores the relationship between two or more Hubs, and nothing else. Relationships are tracked independently of the entities they connect. A new business relationship becomes a new Link; existing Hubs and Satellites remain untouched.
- **Satellite** — Stores the descriptive, frequently changing attributes of a Hub or a Link. Each row carries a load timestamp; new versions are appended rather than overwritten, so the full attribute history accumulates as a natural consequence of the loading pattern.

Two further layers govern data governance Varga, 2014:

- **Raw Vault** — Captures source data exactly as received, applying no business logic. It is the auditable, unmodified source-of-record.
- **Business Vault** — Holds derived constructs — computed satellites, bridge tables, interpreted business rules — isolated from the raw layer so that rule changes never rewrite source history.

Consumers do not query either vault layer directly. A downstream dimensional mart reshapes the vault into the star schemas that reporting tools expect Kimball and Ross, 2013.

2.1.2 Why This Topology Yields Structural Agility

The agility of Data Vault 2.0 follows directly from the decomposition: because keys, relationships, and context are stored separately, structural change is additive. Absorbing a new source means adding hubs, links, and satellites alongside the existing ones, not altering them.

Loading is insert-only. The model never updates or deletes a row; it appends a new version with a fresh timestamp Linstedt and Olschimke, 2015. Full history is therefore a by-product of the loading pattern rather than a feature bolted on through slowly-changing-dimension logic.

Insert-only loading also confines the blast radius of change. A structural change in one source touches only the objects attached to that source, leaving the rest of the model and its downstream consumers untouched. The cost of change scales with the change itself, not with the size of the model.

2.1.3 Operating Principles

Four operating principles turn the topology into a loadable, scalable system.

- **Hash-key surrogates** — Business keys are hashed into fixed-length surrogate keys so that loads can compute keys independently and in parallel, without lookups against previously loaded data Linstedt and Olschimke, 2015. This decouples the loading of each object and lets ingestion scale horizontally.
- **Insert-only loading** — Loads only append. The absence of updates and deletes removes lock contention, makes loads idempotent and restartable, and yields a complete audit trail by construction Vassiliadis, 2009.
- **Raw Vault / Business Vault separation** — The raw layer stays free of business logic so it can serve as the auditable source-of-record; derived logic is quarantined in the Business Vault, so a change to a business rule never rewrites raw history Varga, 2014.
- **Downstream consumption mart** — The vault is optimized for loading and historization, not for querying. A dimensional mart materializes the user-facing layer, accepting query-time cost in exchange for load-time agility Kimball and Ross, 2013.

2.1.4 DV2.0 Assessed Against Structural Scalability

Mapped onto the five C1 structural dimensions, Data Vault 2.0 scores strongly on four and only partially on one. Table 2.1 records the assessment; the single partial score, volume, reflects the model's reliance on the underlying platform rather than any property of the modeling discipline itself.

C1 Dimension	DV2.0 Score	Evidence
Source integration	Strong	A new source maps to new hubs, links, and satellites; existing objects are untouched.
Schema evolution	Strong	Changes are additive and version-aware; no destructive rewrites of existing structures.
Volume and throughput growth	Partial	Hash keys and insert-only loading enable parallel ingestion, but raw scaling capacity is inherited from the host platform.
Historization and auditability	Strong	Insert-only loading retains complete, timestamped history as a by-product of the load pattern.
Loosely-coupled change propagation	Strong	Separation of keys, relationships, and context confines the blast radius of any structural change.

Table 2.1 — Data Vault 2.0 assessed against C1 structural-scalability dimensions. Scores are author assessments. *Author-generated.*

2.1.5 Documented Drawbacks

The same separation that buys structural agility imposes design and implementation complexity. Modeling an entity across hubs, links, and satellites multiplies the number of objects, and getting the decomposition right demands skill that is scarce in practice. Adoption surveys report the skills requirement (48%) and implementation complexity (35%) as the leading barriers AnalyticsCreator, 2025 (*industry source; not peer-reviewed*).

Query performance is the second cost. Reconstructing a single business view means joining a hub to its links and satellites, so analytical queries become join-heavy and slow on the vault itself; the same surveys flag query performance (32%) as a recurring complaint AnalyticsCreator, 2025. The downstream mart exists precisely to absorb this cost, which is why vault data is never user-ready on its own.

These drawbacks are not incidental. The join-heavy query profile and the mandatory consumption layer become structurally important when Data Vault 2.0 is composed with a file-based Lakehouse, whose performance characteristics differ sharply from the relational engines the model assumes. That composability question is examined in Section 2.3.

The standard is also a living one. Recent enhancements relax some of these costs: a virtualized load-end-date removes a maintenance-heavy update from satellite loading, and hybrid architectures push staging and hash-key computation onto a data lake before populating the vault Giebler et al., 2019. These moves already point toward the Lakehouse, and they motivate the analytical and compositional analysis that follows.

2.2 Analytical Integration Patterns

Section 2.1 assessed Data Vault 2.0 on the structural axis. This section turns to the analytical axis and examines how a modern Lakehouse pipeline integrates predictive capability Armbrust et al., 2021. Three patterns define that integration: the order in which data is transformed, the way models are embedded in the pipeline, and the mechanisms that keep data fresh. Each is mapped onto the C1 analytical-scalability dimensions, and the limits are kept in view for the composability analysis that follows.

2.2.1 From ETL to ELT

The first pattern is a reordering of the pipeline. The classical warehouse transforms data *before* loading it (ETL), because storage was expensive and the target schema was fixed in advance Vassiliadis, 2009. The Lakehouse inverts this: it loads raw data first and transforms it *in place* (ELT), because object storage is cheap and compute can be pushed down to the data Harby and Zulkernine, 2024.

Aspect	ETL (classical DW)	ELT (Lakehouse)
Transform location	Separate staging engine, before load	In-platform, after load (pushdown compute)
Storage assumption	Expensive; schema-on-write	Cheap object store; schema-flexible
Raw data	Discarded after transformation	Retained in the Bronze layer
Schema timing	Fixed before ingestion	Applied progressively (Silver, Gold)
Structural agility	Low; a new field needs pipeline rework	High; land first, transform later

Table 2.2 — ETL versus ELT in the modern data pipeline. *Author-generated.*

ELT is what makes the medallion layering of Chapter 1 possible: raw data lands in Bronze, is refined into Silver, and is served from Gold, all within one governed store Databricks, 2022 (*industry practice; not peer-reviewed*). Transformation becomes incremental rather than a precondition for ingestion.

2.2.2 Native Predictive Integration

The second pattern concerns where predictive models live. The decisive question for analytical scalability is whether models are first-class citizens of the pipeline or external appendages Armbrust et al., 2021.

- **Downstream bolt-on** — Data is exported to a separate machine-learning system. Governance and lineage are lost at the boundary, and the copy used for training drifts from the data used for reporting.
- **Native integration** — Training, serving, and monitoring run on the same governed store. One copy of the data feeds both business intelligence and machine learning, and lineage is preserved end to end.

Native integration assembles the model lifecycle into a closed loop on a single platform Kreuzberger et al., 2023. Figure 2.3 shows the arrangement.

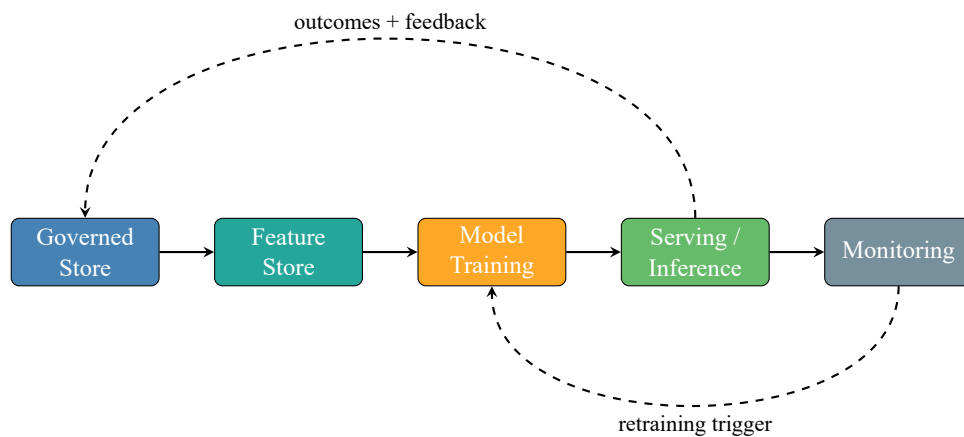


Figure 2.3 — Native predictive pipeline on the Lakehouse. *Author-generated.*

The loop has four working parts Kreuzberger et al., 2023:

- **Feature store** — Curates and serves features consistently to both training and inference, removing training–serving skew.
- **Model training** — Runs on the same governed data, so models train on the trusted production record rather than a stale export.
- **Serving / inference** — Scores new data and writes predictions back to the store, keeping results inside the governed lineage.
- **Feedback and retraining** — Monitoring detects drift and triggers retraining, closing the loop required for prescriptive analytics “Feedback-Driven Decision Support System with Incremental Retraining,” 2025.

This native arrangement is what lets a Lakehouse reach the predictive and prescriptive levels of the maturity spectrum without a downstream bolt-on “AI-Driven Decision Support System: A Review Integrating ML, NLP, and DL for Predictive Analytics,” 2025.

2.2.3 Streaming and Incremental Analytics

The third pattern keeps the pipeline fresh. Prescriptive decisions need data recent enough to act on, which shifts ingestion and scoring from periodic batches toward continuous flow.

- **Change data capture (CDC)** — Streams row-level changes from source systems as they happen, rather than reloading whole tables Kleppmann, 2017.
- **Continuous ingestion** — A single replayable stream feeds both real-time and historical processing, the Kappa pattern Kreps, 2014 (*industry practice; not peer-reviewed*).
- **Near-real-time scoring** — Models score incoming records on arrival, so predictions track the current state of the business.

- **Online learning over drift** — Where distributions shift, models update incrementally on the stream rather than waiting for a full retrain Bifet et al., 2018.

2.2.4 Analytical Integration Assessed Against Analytical Scalability

Mapped onto the five C1 analytical dimensions, the Lakehouse analytics stack scores strongly on four and partially on one. Table 2.3 records the assessment. The single partial score, data readiness, is the pivot of this chapter: the stack delivers freshness natively, but the *trustworthiness* of the data it serves depends on a modeling discipline the Lakehouse does not itself supply.

C1 Dimension	Lakehouse Score	Evidence
Maturity reach	Strong	Supports descriptive through prescriptive analytics on one platform.
Native model integration	Strong	Training, serving, and monitoring run on the governed store as first-class citizens.
Data readiness	Partial	Freshness is native via streaming, but data trust and feature quality depend on upstream modeling discipline.
Workload plurality	Strong	Business intelligence and machine learning are served from the same governed layer.
Lifecycle support	Strong	Feature stores, monitoring, and retraining provide a closed model lifecycle.

Table 2.3 — The Lakehouse analytics stack assessed against C1 analytical-scalability dimensions. Scores are author assessments. *Author-generated*.

The two assessments now sit side by side. Data Vault 2.0 is structurally strong but analytically capped; the Lakehouse analytics stack is analytically strong but assumes the disciplined, trustworthy data it does not produce. The partial scores of each are the strengths of the other. Whether the two actually compose into one architecture is the question of Section 2.3.

2.3 The Core Tension: Composing DV2.0 with the Lakehouse

The previous two sections established a complementarity: Data Vault 2.0 is structurally strong, the Lakehouse analytics stack is analytically strong, and each covers the other's weak dimension. The obvious move is to combine them. This section asks whether they actually compose, or whether one is quietly bent to fit the other. The answer matters, because the

whole thesis rests on the two forming a single coherent architecture rather than an uneasy stack.

2.3.1 The Clash, Stated Precisely

The two paradigms come from different worlds. Data Vault 2.0 is a *relational-lineage modeling discipline*: structured tables, enforced keys, explicit historization, and business views reconstructed through joins Linstedt and Olschimke, 2015. The Lakehouse is *file-based and schema-flexible*: open columnar files, a tendency toward schema-on-read, and transactional guarantees layered on top of files rather than tables Armbrust et al., 2020, 2021.

Property	Data Vault 2.0	Lakehouse
Unit of storage	Structured relational tables	Open files (Parquet, ORC)
Schema posture	Enforced, schema-on-write	Flexible, schema-on-read tendency
Keys	Enforced business and hash keys	No enforced keys at the file layer
Historization	Explicit, modeled in satellites	Table-version time travel
Query model	Join-based reconstruction	Columnar file scan

Table 2.4 — The clash in one view: relational-lineage discipline versus file-based flexibility.
Author-generated.

Table 2.4 frames the question. These are not small differences of configuration; they are different assumptions about what data *is*. Composition is therefore not guaranteed, and has to be examined rather than assumed.

2.3.2 What Industry Practice Shows

In practice, the industry has already chosen a direction: Data Vault 2.0 is implemented *on top of* the Lakehouse storage layer, not as an alternative to it Databricks, 2022 (*industry practice; not peer-reviewed*). The vault tables are stored as Delta tables, ingestion uses auto-loaders, and loading is orchestrated through declarative pipelines. Figure 2.4 shows the canonical placement.

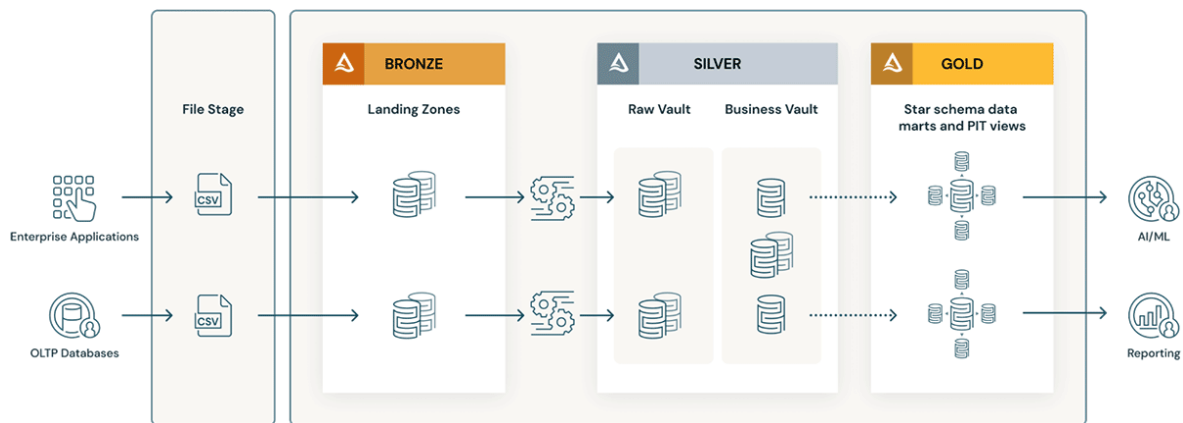


Figure 2.4 — Data Vault 2.0 mapped onto the Lakehouse medallion. Source: Databricks, 2022 (*industry/vendor source; not peer-reviewed*).

The mapping is direct. File staging feeds the Bronze landing zones; the Raw Vault and Business Vault occupy the Silver layer; star-schema marts and point-in-time views are served from Gold. The diagram makes the key fact visible: in current practice the vault is a *tenant* of the Lakehouse, not a competitor to it.

Independent and vendor accounts converge on the same pattern. Hybrid designs push staging and hash-key computation onto the lake before populating the vault Moermans and Olschimke, 2023; Moronta, 2026 (*industry practice; not peer-reviewed*), and a peer-reviewed study reports the same approach for modeling data lakes with Data Vault Giebler et al., 2019.

2.3.3 Where They Align and Where They Conflict

The placement works because real alignments exist, but it strains because real conflicts persist. Table 2.5 pairs them by theme; reading each row across exposes the trade-off the combination must manage.

Theme	Where they align ✓	Where they conflict ×
Storage & compute	Decoupled storage and compute serve DV2.0's parallel loading.	Join-heavy reconstruction runs against the grain of columnar file scans.
Historization	Insert-only loading maps cleanly onto Delta time travel.	Two history mechanisms overlap: satellite timestamps and table versions.
Integration	Both are built to absorb heterogeneous sources.	Schema enforcement and lineage discipline fight the lake's schema-on-read habit.
Governance	The Business Vault centralizes derived business rules.	The lake's "land everything first" ethos invites ungoverned sprawl.
Skills	—	Modeling rigor adds a skills cost on top of the lake's low entry barrier.

Table 2.5 — Alignments and conflicts between Data Vault 2.0 and the Lakehouse, paired by theme. *Author-generated.*

The sharpest conflict is performance. Data Vault 2.0 reconstructs a business view by joining a hub to its links and satellites, and that join-heavy profile is precisely the workload columnar file scans handle worst. The Gold-layer mart absorbs the cost, but its existence is an admission that the vault itself is not queryable at interactive speed on the lake.

The governance conflict is subtler and just as real. The vault imposes discipline; the lake removes friction from landing raw data. A combined architecture has to hold both at once, and nothing in either paradigm forces that balance — it has to be designed in.

2.3.4 What the Literature Does Not Resolve

The decisive observation of this chapter is what is *missing*. The composition is asserted and practiced, but not methodically evaluated.

- **No peer-reviewed evaluation** — The combination is documented almost entirely in vendor blogs and practitioner guides, not in peer-reviewed studies Databricks, 2022; Moermans and Olschimke, 2023; Moronta, 2026.
- **Undocumented trade-offs** — The performance and governance costs above are acknowledged informally but rarely measured or modeled in the academic literature.
- **Surveys stop short** — Recent peer-reviewed surveys of the Lakehouse map its capabilities but do not assess its composition with a formal modeling discipline like DV2.0 Harby and Zulkernine, 2024; Schneider et al., 2024.

This unresolved state is itself a finding. The industry has run ahead of the evidence, combining the two paradigms by convention rather than by demonstrated principle. That gap — a practiced combination without a reasoned framework — is the opening this thesis addresses. Section 2.4 turns the alignments and conflicts above into the explicit criteria that Chapter 3 uses to build and assess that framework.

2.4 A Criteria Grid Derived from the Analysis

This section delivers construct **C2**: a set of six evaluation criteria distilled from the analysis of §§2.1–2.3. The criteria are not chosen freely. Each is forced by a finding already established in this chapter, and each comes with an explicit rubric so that the comparison in Chapter 3 reads as assessment rather than opinion.

2.4.1 Where the Criteria Come From

The six criteria trace directly to the three preceding sections. Figure 2.5 shows the provenance.

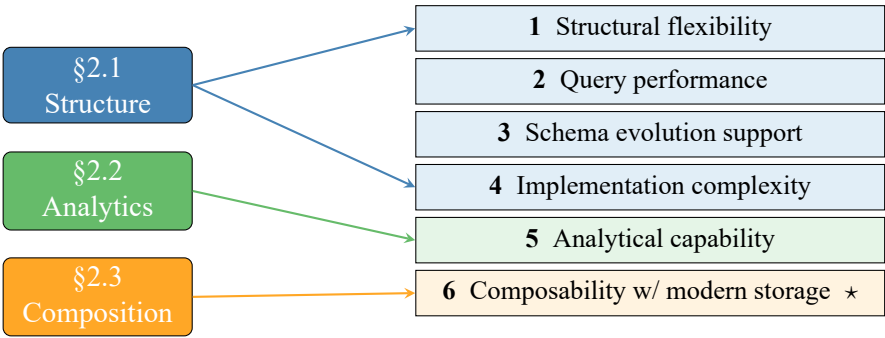


Figure 2.5 — Provenance of the C2 criteria. The ★ marks the author’s added criterion.
Author-generated.

Criterion 6 is the author’s refinement. The composability tension of §2.3 is not part of the classical comparison vocabulary; promoting it to a first-class criterion is what lets Chapter 3 evaluate the combination directly rather than the two paradigms in isolation.

2.4.2 The Six Criteria

Table 2.6 defines each criterion and links it to the dual-scalability dimensions of construct C1.

#	Criterion	Definition	Linked C1 dimension
1	Structural flexibility	Ease of absorbing new, heterogeneous sources.	Source integration
2	Query performance	Efficiency of data retrieval at consumption time.	(cross-cutting)
3	Schema evolution support	Cost and blast radius of changing existing structures.	Schema evolution; change propagation
4	Implementation complexity	Skills, time, and cost to build and operate.	(cross-cutting)
5	Analytical capability	Maturity reach and native model integration.	Maturity reach; native model integration
6	Composability with modern storage	Degree to which the model composes with file-based Lakehouse storage without distortion.	(author's refinement; from §2.3)

Table 2.6 — Construct C2 — the six evaluation criteria and their C1 anchors. *Author-generated.*

2.4.3 Assessment Rubric

Each criterion is scored on a three-level rubric with stated meanings, so that a score in Chapter 3 can be defended against the evidence. Table 2.7 gives the descriptors.

Criterion	Low	Medium	High
Structural flexibility	New sources force redesign	Sources absorbed with effort	New sources added without touching existing structures
Query performance	Heavy joins; slow interactive queries	Acceptable after tuning or a mart	Fast interactive queries on the served layer
Schema evolution support	Changes ripple destructively	Changes possible but costly	Additive, version-aware, isolated
Implementation complexity	Specialized skills; long build	Moderate skill and time	Simple; few skills; fast
Analytical capability	Descriptive only	Predictive with a bolt-on	Prescriptive, natively integrated
Composability with modern storage	Paradigms clash; one is distorted	Composable with workarounds	Composes cleanly on shared storage

Table 2.7 — C2 assessment rubric. For every criterion the **High** column is the favourable end, including implementation complexity, where High denotes low burden. *Author-generated.*

Together, Table 2.6 and Table 2.7 are the instrument this chapter contributes. Chapter 3 applies them to Data Vault 2.0, the Lakehouse, and the proposed combined framework, producing the comparative evaluation that answers the composability question (RQ3).

Conclusion

This chapter set the two candidate components against each other. Data Vault 2.0 is structurally agile but analytically capped and costly to query; the Lakehouse analytics stack is the mirror image, natively predictive but reliant on a data discipline it does not supply. Their strengths are complementary, and the industry already combines them, placing the vault on top of the Lakehouse, yet that combination rests on vendor practice rather than peer-reviewed evidence.

That gap is what construct C2 is built to close. Its six rubric-backed criteria turn the alignments and conflicts of this chapter into an instrument, which Chapter 3 applies to design and evaluate a combined reference framework.

Chapter 3

Proposed Reference Framework and Analytical Evaluation

Introduction

Chapter 2 closed on a sharpened question rather than an answer. Its composability analysis (§2.3) showed that Data Vault 2.0 and the Lakehouse do combine in current practice, but that the combination rests on an interface neither paradigm specifies: the modeling discipline assumes relational lineage and hash-keyed history, while the analytics stack assumes file access and schema-on-read. The chapter also produced the instrument needed to judge any proposed resolution — the six-criteria evaluation grid (§2.4). This chapter uses both outputs: it builds an architecture that names and governs that interface, then evaluates it with the grid.

This is the construct chapter, and it carries C3. A theoretical thesis is most exposed here, because a jury can attack a proposed architecture on two fronts. The first is specification: a block diagram with boxes and arrows but no component definitions and no stated contract is a picture, not an artifact. The second is evaluation: a comparison table with scores but no stated method reads as opinion, not assessment. This chapter is organized to close both fronts deliberately.

It closes the first front by presenting the framework as a formal artifact. Section 3.1 derives the requirements from RQ1–RQ3 and the dual-scalability dimensions of §1.5, so that every design choice traces to a stated need. Section 3.2 then specifies a layered reference model with named components and their responsibilities, an explicit integration contract between the agile modeling layer and the native analytics layer (§3.2.3), and a set of numbered design invariants (§3.2.4) that can be probed one by one. The contract is the centerpiece: it states what the modeling layer guarantees to the analytics layer, and it scopes explicitly what it does not solve.

It closes the second front by stating the evaluation method before applying it. Section 3.3 defines the assessment as conceptual and criteria-based, explicitly non-empirical, with a scoring rubric and an evidence rule that admits no bare scores. Section 3.4 then applies that method across Kimball, Inmon, Data Vault 2.0, and the proposed framework, conceding

where the framework inherits Data Vault 2.0’s complexity cost rather than claiming it wins everywhere. Section 3.5 examines the trade-offs, the conditions under which the architecture earns its complexity, and the limits of a validation that remains conceptual.

3.1 Requirements and Design Rationale

The framework is not designed freely. Its requirements are derived from the research questions of the introduction and the ten dual-scalability dimensions formalized as construct C1 (§1.5), so that every later design decision answers a stated need rather than a preference. This section makes that derivation explicit before any architecture is drawn. Figure 3.1 shows the derivation path, and Table 3.1 states each requirement with the question it answers and the dimension it serves.

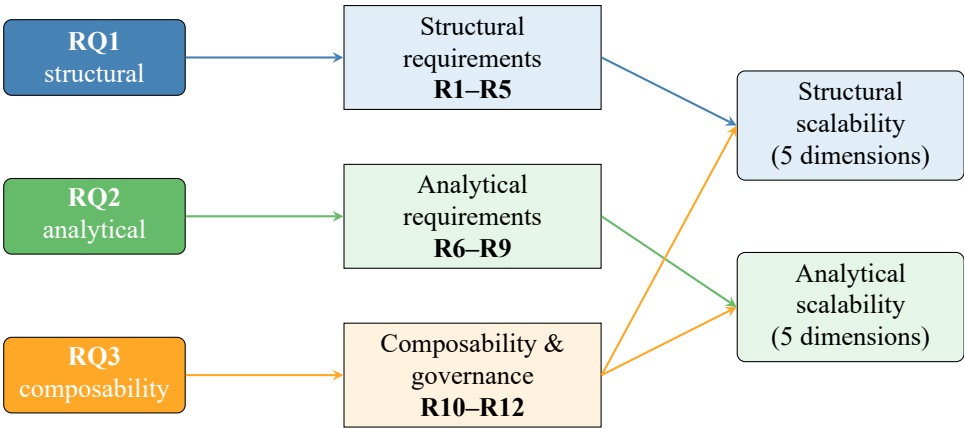


Figure 3.1 — Requirements derivation. Each research question forces a block of requirements, which in turn serve the C1 scalability dimensions. RQ3 is the only question that binds both axes — the composability requirements (R10–R12) are what couple structure to analytics. *Author-generated.*

Figure 3.1 carries the central design claim of the chapter: the structural and analytical requirement blocks satisfy their respective axes in isolation, but RQ3 is discharged only by the cross-cutting requirements R10–R12, which is precisely the composability tension of §2.3 restated as a design obligation.

ID	Requirement	RQ	C1 dimension served
R1	Source onboarding is non-destructive: a new heterogeneous source is absorbed by adding objects, never by restructuring existing ones Linstedt and Olschimke, 2015.	RQ1	Source integration
R2	Schema evolution is additive and version-aware: structural change creates new structures rather than mutating in place Kleppmann, 2017.	RQ1	Schema evolution
R3	Data volume and ingestion load scale without architectural redesign Armbrust et al., 2020.	RQ1	Volume / throughput
R4	Every entity retains a complete, non-destructive, auditable history Linstedt and Olschimke, 2015.	RQ1	Historization
R5	A structural change in one source or layer is isolated from downstream consumers Linstedt and Olschimke, 2015.	RQ1	Change propagation
R6	Predictive and prescriptive models are first-class citizens of the pipeline, not downstream bolt-ons Armbrust et al., 2021.	RQ2	Maturity reach; Native model integration
R7	The analytical consumption point exposes fresh, trusted, feature-ready data Kreuzberger et al., 2023.	RQ2	Data readiness
R8	Business-intelligence and machine-learning workloads are served from one governed layer without duplication Schneider et al., 2024.	RQ2	Workload plurality
R9	Model retraining, feedback loops, and model governance occur inside the pipeline Kreuzberger et al., 2023.	RQ2	Lifecycle support
R10	The modeling and analytics layers interface through an explicit integration contract over historized, business-keyed, lineage-stamped entities Jones, 2023.	RQ3	Source integration ↔ Native model integration
R11	A single open table format mediates relational-lineage discipline and machine-learning file access Armbrust et al., 2020; Databricks, 2022.	RQ3	Schema evolution; Workload plurality
R12	Lineage and governance span all layers end to end Giebler et al., 2019.	RQ1–RQ3	Historization; Data readiness

Table 3.1 — Requirements traceability: each requirement maps to the research question that forces it and the C1 scalability dimension it serves. Blocks are color-coded by axis — structural (blue), analytical (green), composability and governance (orange). *Author-generated.*

Two properties of Table 3.1 matter for what follows. First, the twelve requirements cover all ten C1 dimensions, so the framework is answerable against the same instrument used to expose the gap in Chapter 1. Second, the structural block (R1–R5) and the analytical block (R6–R9) describe capabilities that Data Vault 2.0 and the Lakehouse already provide individually, as Chapter 2 established; the design work is concentrated in R10–R12, the requirements no single paradigm satisfies alone. These three are the obligations the integration contract and governance plane of §3.2 must discharge.

3.2 The Reference Framework

3.2.1 Layered Reference Model

The framework is organized as four stacked functional layers resting on a shared storage substrate and wrapped by one cross-cutting governance plane. The ordering follows the data path: heterogeneous sources are captured at the bottom, integrated and historized in the middle, and consumed at the top by two parallel paths — business intelligence and native analytics. The single design decision that distinguishes this arrangement from a conventional warehouse is the placement of the analytics layer: it reads from the agile modeling layer directly, not from the serving marts downstream of it. Figure 3.2 shows the model.

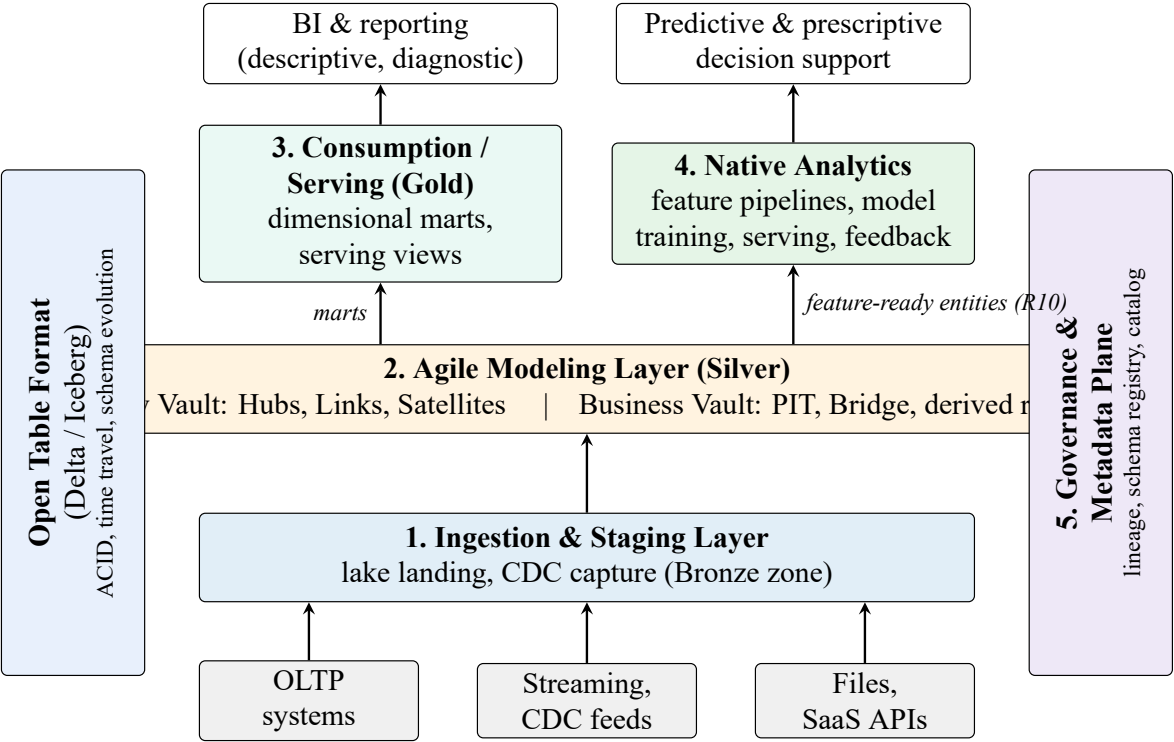


Figure 3.2 — The layered reference framework. Data flows bottom-up; the native analytics layer (4) reads from the agile modeling layer (2) directly, not through the serving marts (3). The open table format is the shared physical substrate; the governance plane cross-cuts every layer.
Author-generated.

The arrangement answers the two scalability axes with distinct layers and couples them through one substrate. Layer 2 carries structural scalability (Data Vault 2.0 mechanics); layer 4 carries analytical scalability (native model integration); the open table format lets both read the same physical data without a separate copy, and the governance plane makes lineage and schema evolution observable across the whole stack. The direct modeling-to-analytics edge is the structural expression of requirement R10 — the interface specified in §3.2.3.

3.2.2 Component Definitions

Each layer decomposes into named components. Table 3.2 states, for every component, its inputs, outputs, responsibility, and the C1 dimension it serves. The input/output columns are what make the diagram an artifact rather than a sketch: they define the interfaces along which the layers connect.

Component	Inputs	Outputs	Responsibility	Dimension
Layer 1 — Ingestion & Staging				
Source connectors / CDC	Source systems	Change records	Capture heterogeneous sources incrementally	Source integration
Lake landing (Bronze)	Change records	Immutable landed files	Persist raw data, schema-tolerant	Volume / throughput
Layer 2 — Agile Modeling				
Raw Vault	Landed files	Historized, keyed entities	Non-destructive integration and historization	Schema evolution; Historization
Business Vault	Raw Vault	PIT, Bridge, derived entities	Centralize business rules; pre-assemble history	Change propagation; Data readiness
Layer 3 — Consumption / Serving				
Dimensional marts / serving views	Business Vault	Star schemas, views	Serve interactive BI queries	Workload plurality
Layer 4 — Native Analytics				
Feature pipelines	Business / Raw Vault	Versioned feature sets	Derive features from Satellite attributes	Native model integration; Data readiness
Model training & serving	Feature sets	Predictions, scores	Train and serve predictive / prescriptive models	Maturity reach
Lifecycle manager	Predictions, outcomes	Retraining triggers, registry entries	Feedback loops, retraining, model governance	Lifecycle support
Layer 5 — Governance & Metadata Plane (cross-cutting)				
Schema registry	Schema changes	Versioned, compatibility-checked schemas	Propagate evolution non-destructively	Schema evolution; Change propagation
Catalog & lineage tracker	All layer operations	Lineage graph, catalog	Maintain end-to-end lineage	Historization; Data readiness
Open table format runtime	All reads / writes	ACID tables, time travel	Mediate relational and file access on one copy	Workload plurality; Volume

Table 3.2 — Component definitions across the five layers: inputs, outputs, responsibility, and the C1 dimension each serves. *Author-generated.*

The components in layers 1–3 are standard for a Data Vault 2.0 deployment on a Lakehouse Databricks, 2022; Linstedt and Olschimke, 2015. Layer 4 and the schema registry are where the framework adds structure: the feature pipelines read Vault entities rather than raw files, and the registry turns Data Vault 2.0’s additive-change convention into an enforced compatibility guarantee. How those components interoperate across the layer 2–layer 4 boundary is the integration contract, specified next.

3.2.3 The Integration Contract

The integration contract is the centerpiece of the framework. Chapter 2 established that Data Vault 2.0 and the Lakehouse are practiced together but their interface is specified by neither paradigm (§2.3); this contract specifies it. It is a one-way agreement: the agile modeling layer (provider) guarantees a set of properties on the entities it exposes, and the native analytics layer (consumer) builds features only on those guarantees. The construct borrows the data-contract pattern from current data engineering practice Jones, 2023 (*industry practice; not peer-reviewed*) and binds it to Data Vault 2.0 semantics. Figure 3.3 frames the interface.

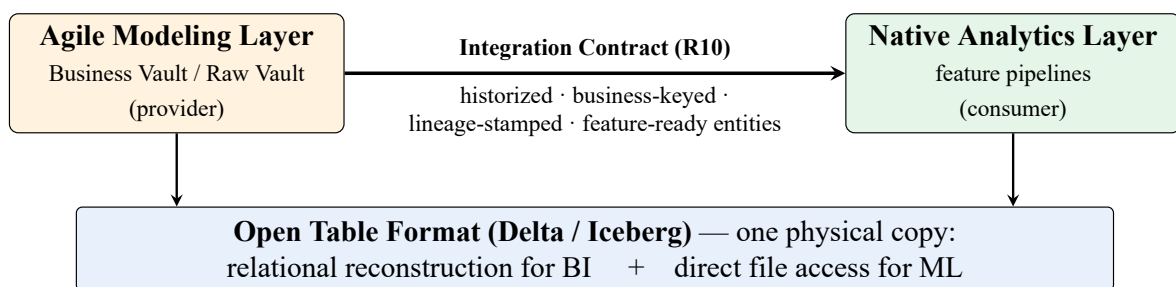


Figure 3.3 — The integration contract as a provider–consumer interface over a shared storage substrate. The modeling layer guarantees four entity properties; the analytics layer relies only on them. *Author-generated.*

Contract clauses

Table 3.3 states the contract as six clauses. Each pairs a provider guarantee with the mechanism that realizes it and the requirement it discharges.

#	Provider guarantee	Mechanism	Serves
G1	Historized entities	Satellite <code>load_dts</code> gives point-in-time reconstruction of any attribute	R4
G2	Business-keyed identity	Stable hash business keys keep joins valid across schema change	R1, R5
G3	Lineage stamp	Every entity carries source Satellite and load timestamp, recorded by the catalog	R12
G4	Feature-ready shape	Business Vault PIT and Bridge structures pre-assemble history, so features need no re-derivation	R7
G5	Schema-evolution compatibility	Additive change only (new Satellites/columns); schema registry enforces backward compatibility	R2
G6	Format-access duality	Open table format exposes the same entities as governed tables and as files	R8, R11

Table 3.3 — The integration contract in six clauses: each provider guarantee, its realizing mechanism, and the requirement it discharges. *Author-generated.*

Three mechanics make the clauses operational. *Feature derivation*: features are computed from Satellite attributes and Business Vault entities, never from raw staging, and point-

in-time joins key on `load_dts` (G1, G4). *Schema-evolution propagation*: an additive change creates a new Satellite or column, the schema registry issues a new compatible version, and feature pipelines opt into it explicitly; because no attribute is mutated in place, a model trained on an earlier version keeps reading valid inputs (G5). *Format mediation*: the open table format stores each entity once and serves join-based reconstruction for BI and columnar file scans for machine learning from that single copy, so the two workloads never diverge into separate datasets Armbrust et al., 2020, 2021 (G6).

Scope and stated assumptions

The contract resolves part of the §2.3 tension and scopes the rest explicitly. Stating the limits is deliberate: an honest boundary is more defensible than an overclaimed resolution.

- **A1 (single format).** The contract assumes one open table format across all layers. It does not address cross-format federation; mixing Delta and Iceberg is out of scope.
- **A2 (performance, conceded).** The contract does not eliminate the join-heavy cost of Vault reconstruction identified in §2.3; it confines it. The Business Vault and serving marts absorb the cost for BI, and feature pipelines that read the Raw Vault directly inherit it. This is a real limitation, not a solved problem.
- **A3 (history reconciliation).** Two history mechanisms coexist — Satellite `load_dts` and table-version time travel. The contract makes `load_dts` the system of record for analytical point-in-time correctness; table versions are treated as operational metadata only. This removes the ambiguity flagged in Table 2.5.
- **A4 (interface only).** The contract governs the modeling-to-analytics interface. It does not specify model internals, serving latency, or BI tooling, which remain implementation choices.

3.2.4 Design Rules and Invariants

The contract is enforced by a small set of design invariants. Each is a property the framework must hold at all times, stated so that it can be checked — and contested — one at a time. The bracketed labels map each invariant to the requirement (R) or contract clause (G) it enforces.

- I1.** Every analytical feature traces to a historized Satellite attribute; no feature originates outside the modeling layer. [R4, G1]
- I2.** The native analytics layer reads only from the modeling or serving layers, never from raw staging directly. [R10]
- I3.** Schema changes in the Raw Vault are additive — new Satellites or columns; destructive changes are prohibited. [R2, G5]

- I4.** Business keys are stable and source-system-independent (hash keys). [R1, G2]
- I5.** Every entity entering the analytics layer carries a lineage stamp: source Satellite plus load timestamp. [R12, G3]
- I6.** Point-in-time correctness uses Satellite `load_dts` as the temporal system of record, not table-version time travel. [A3]
- I7.** Business intelligence and machine learning read the same open-format tables; no parallel analytical copy is maintained. [R8, G6]
- I8.** Every deployed model is pinned to a feature-set schema version; an additive schema change cannot silently alter a deployed model’s inputs. [R9, G5]

Invariants I1, I2, and I5 close the provenance of every analytical result back to a governed source; I3, I6, and I8 are what let schema evolution and model training coexist without breaking each other — the property the literature reviewed in §2.3 leaves unspecified.

3.3 Evaluation Methodology

The comparison that follows is a conceptual, criteria-based assessment, not an experiment. The thesis builds no prototype, so each approach is judged against documented evidence rather than measured performance. This is a stated limit, not a concealed one: the method establishes *relative* standing among approaches under a fixed rubric, not absolute metrics. Defining the method before applying it is what separates the comparison in §3.4 from an opinion table. Figure 3.3 shows the protocol.

Vault 2.0–Lakehouse composition — and are flagged in-text as not peer-reviewed. A rating that rests only on vendor evidence is treated as weaker than one grounded in peer-reviewed work, and the discussion (§3.5) revisits the ratings most exposed to this gap.

Threats to validity. The method has known weaknesses, stated up front in Table 3.4 so that the comparison is read with them in view rather than despite them.

Threat	Description	Mitigation
T1 — Rating subjectivity	Ordinal scores reflect author judgment.	The rubric fixes the meaning of each level; every rating is cited or argued.
T2 — Literature gaps	Composability evidence is thin and largely non-peer-reviewed.	Gaps are flagged; affected claims are scoped, not asserted (assumption A2).
T3 — Vendor-source bias	Industry sources may overstate their own products.	Vendor sources are flagged, used for practice not principle, and cross-checked against peer-reviewed work where it exists.
T4 — Self-assessment bias	The author rates a framework of their own design.	The framework is not permitted to win every criterion; it inherits Data Vault 2.0’s complexity cost explicitly (§3.4).
T5 — No empirical validation	Ratings predict standing; they do not measure it.	Stated as the central limitation; claims are relative, not absolute.

Table 3.4 — Threats to the validity of the evaluation, with mitigations. *Author-generated.*

With the method, instrument, scoring scale, evidence rule, and threats stated, the comparison can proceed. Section 3.4 applies this protocol to Kimball, Inmon, Data Vault 2.0, and the proposed framework.

3.4 Comparative Study

Applying the protocol of §3.3 to the six criteria across the four approaches produces the matrix in Table 3.5. The justification that follows is organized by criterion cluster, not cell by cell, and each cluster cites the evidence behind its ratings.

Criterion	Kimball	Inmon	DV2.0	Proposed
1. Structural flexibility	Low	Low	High	High
2. Query performance	High	Medium	Low	Medium
3. Schema evolution support	Low	Low	High	High
4. Implementation complexity	Medium	Low	Low	Low
5. Analytical capability	Low	Low	Low	High
6. Composability with modern storage	Medium	Low	Medium	High

High Medium Low favourable to unfavourable. For implementation complexity, **High** denotes low burden.

Table 3.5 — Evaluation matrix: four approaches against the six C2 criteria, scored on the rubric of Table 2.7. The proposed framework leads on structure, analytics, and composability, but not on query performance or implementation complexity. *Author-generated.*

Structural criteria (1, 3). The classical paradigms score Low on both structural flexibility and schema evolution. Kimball’s conformed dimensions and Inmon’s normalized core each turn a new source or a structural change into a redesign event Inmon, 2005; Kimball and Ross, 2013. Data Vault 2.0 and the proposed framework score High on both: the Hub–Link–Satellite decomposition absorbs new sources and structural change additively Linstedt and Olschimke, 2015. The proposed framework inherits this directly, and its schema registry (invariant I3) hardens Data Vault 2.0’s additive convention into an enforced compatibility guarantee.

Query performance (2). Performance inverts the ranking. Kimball scores High — the star schema is built for fast interactive queries Kimball and Ross, 2013. Data Vault 2.0 scores Low: business views are reconstructed through hub–link–satellite joins, the workload columnar file scans handle worst (§2.3). Inmon and the proposed framework sit at Medium; both interpose a serving layer — Inmon’s data marts, the framework’s Gold marts and Business Vault PIT/Bridge structures — that absorbs the join cost for BI without removing it. The framework does not claim Kimball’s performance; assumption A2 concedes the residual cost explicitly.

Implementation complexity (4). This is where the proposed framework is weakest, and the matrix says so. Inmon’s enterprise normalized core, Data Vault 2.0’s modeling rigor and specialized skills, and the proposed framework all score Low — high burden. The framework is arguably the most demanding of the four: it carries Data Vault 2.0’s full modeling cost and adds a native analytics layer, a governance plane, and the integration contract. Kimball scores Medium, reflecting its comparatively faster delivery of business intelligence Kimball and Ross, 2013. A framework that scored High here would not be credible.

Analytical capability (5). Only the proposed framework reaches High. Kimball, Inmon, and Data Vault 2.0 are descriptive-to-diagnostic by construction and score Low; none integrates predictive or prescriptive capability natively, as Chapter 2 established Inmon, 2005; Kimball and Ross, 2013; Linstedt and Olschimke, 2015. The framework’s native analytics layer, reading feature-ready entities across the integration contract, lifts it to prescriptive reach with models as first-class pipeline citizens Armbrust et al., 2021; Kreuzberger et al., 2023.

Composability with modern storage (6). Composability separates the framework from its parent. Data Vault 2.0 scores Medium: it runs on the Lakehouse, but by workaround — vendor practice places the vault on Delta storage and absorbs the join cost downstream Databricks, 2022; Moronta, 2026 (*industry practice; not peer-reviewed*), with a peer-reviewed account confirming the pattern for data lakes Giebler et al., 2019. Inmon scores Low and Kimball Medium; neither was designed for file-based storage. The proposed framework scores High because composition is designed in rather than retrofitted: a single open table format and the integration contract make the modeling and analytics layers share one physical copy (invariant I7). This is the rating most exposed to the evidence gap of §2.3, and §3.5 returns to it.

Read across, the matrix has a clear shape. The proposed framework leads on the structural, analytical, and composability criteria, matches the field on query performance, and pays for this on implementation complexity. That trade-off — capability bought with build cost — is the subject of the discussion that follows.

3.5 Discussion

The trade-off the framework makes. The matrix of §3.4 shows a single coherent bargain rather than a uniform advantage. The framework buys structural agility, native analytical capability, and clean composability, and it pays in implementation complexity and a query-performance ceiling below Kimball’s. The trade is deliberate. Structural and analytical scalability are the two axes the introduction set out to satisfy jointly (RQ1, RQ2); query performance is recoverable at the serving layer, and implementation complexity is a one-time build cost. An organization that needs neither axis should not pay that cost — which is the question of applicability.

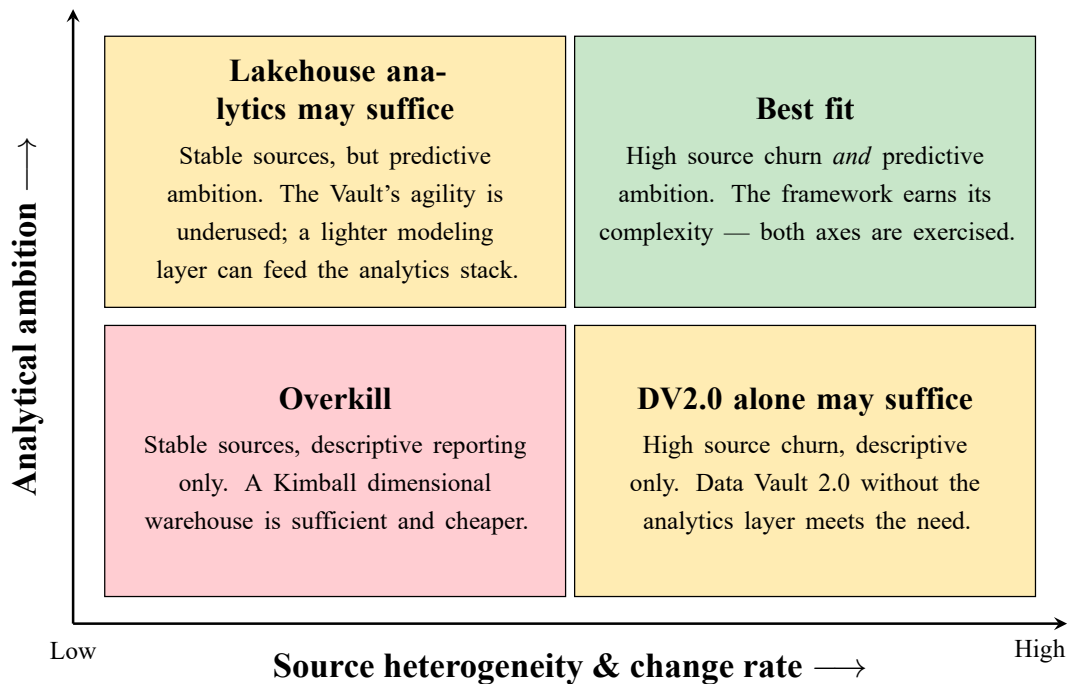


Figure 3.4 — Conditions of applicability. The framework is worth its complexity in the high–high quadrant; in the other three, a simpler architecture meets the need at lower cost. *Author-generated.*

When the framework is worth its cost. Figure 3.4 maps the decision against two factors. The framework is justified only in the top-right quadrant: high source heterogeneity and change rate, combined with ambition beyond descriptive reporting. There, both scalability axes are exercised and the build cost is repaid. In the bottom-left quadrant — stable sources, descriptive reporting — a Kimball dimensional warehouse is sufficient and far cheaper, and the framework is overkill Kimball and Ross, 2013. The two off-diagonal quadrants are partial: with stable sources but predictive ambition, the Vault's structural agility is underused; with high churn but no analytical ambition, Data Vault 2.0 alone meets the need without the analytics layer Linstedt and Olschimke, 2015. The framework does not claim universal applicability, and that boundary is part of the contribution.

Threats to validity, revisited. The threats stated in Table 3.4 bear unevenly on the results. The structural and analytical ratings rest on foundational, peer-reviewed sources and are the most secure. The composability rating (criterion 6) is the most exposed: it is the one most dependent on industry and vendor evidence (threat T3) and on a literature that does not yet evaluate the composition formally (threat T2). The self-assessment threat (T4) is contained by the matrix itself, which the framework does not sweep. None of these threats is dissolved here; they qualify the strength of the claims rather than the claims themselves.

Limitations. Three limits follow directly and are stated plainly.

- **No empirical validation.** The framework is evaluated conceptually, not measured. Its

ratings predict relative standing; they are not benchmarks. Empirical validation is the principal direction for future work.

- **Composability evidence is thin.** The central composability claim leans on vendor practice and a small peer-reviewed base Databricks, 2022; Giebler et al., 2019. Until the composition is studied empirically, that claim is scoped, not settled.
- **Reference design, not an implementation.** The framework specifies layers, a contract, and invariants; it deliberately leaves engine choice, serving latency, and tooling open (assumption A4). It is a blueprint to instantiate, not a system to deploy as-is.

Conclusion

This chapter delivered construct C3. It specified a layered reference framework as a formal artifact, not a block diagram: four functional layers on a shared open-format substrate, component definitions with stated inputs and outputs, an integration contract of six clauses between the agile modeling layer and the native analytics layer, and eight design invariants that make the contract checkable. It then evaluated the framework with the C2 instrument under a stated method, conceding where the framework is weak — query performance and implementation complexity — rather than claiming a clean sweep.

The framework addresses all three research questions, each through a specific element of the design. Table 3.6 states how.

RQ	How the framework addresses it	Framework element
RQ1	New sources, schema changes, and volume growth are absorbed additively by the agile modeling layer, with destructive change prohibited.	Layers 1–2; requirements R1–R5; invariants I3, I4
RQ2	Predictive and prescriptive models are first-class pipeline citizens, reading feature-ready entities rather than being bolted on downstream.	Layer 4; requirements R6–R9; clauses G1, G4
RQ3	The two paradigms compose through an explicit contract over a single open table format, under stated conditions of applicability.	Integration contract (G1–G6); invariant I7; §3.5

Table 3.6 — How Chapter 3 addresses the three research questions. *Author-generated.*

These are the answers the construct supports, not final claims. The General Conclusion takes Table 3.6 as its starting point: it answers RQ1, RQ2, and RQ3 explicitly against the evidence assembled across all three chapters, restates the limitations that qualify each answer, and sets out the empirical validation that this theoretical thesis leaves to future work.

General Conclusion

Recall of the Problem and Approach

Classical decision support architectures force two premature commitments: a fixed structural schema, expensive to revise once ingestion begins, and a fixed analytical scope, typically descriptive reporting that cannot be extended toward prediction without grafting external systems onto the pipeline. The literature treats these as separate problems — structural agility through Data Vault 2.0, analytical integration through the Lakehouse — and never frames both as dimensions of one property. This thesis treated them as a single problem of *dual scalability* and proceeded in three steps that map to its three chapters: conceptualize the property (C1, Chapter 1), analyze whether the two leading paradigms compose (C2, Chapter 2), and construct and evaluate a reference framework that unifies them (C3, Chapter 3).

Constructs Against the Research Questions

RQ1 — structural. *How can a DSIS architecture absorb new sources, schema changes, and volume growth without disruptive re-engineering?* It does so when structural change is made additive rather than destructive. Construct C1 defined structural scalability through five dimensions; the framework’s agile modeling layer (§3.2) discharges them with the Data Vault 2.0 Hub–Link–Satellite decomposition on an open table format, where a new source is absorbed by adding objects, not by restructuring existing ones (requirements R1–R5). Invariants I3 and I4 make this binding: schema changes are additive and business keys are stable, so growth in sources, structure, and volume does not propagate as a rebuild.

RQ2 — analytical. *How can predictive/prescriptive capability be integrated natively into the decision pipeline rather than bolted on downstream?* By positioning the analytics layer to read from the modeling layer directly. Construct C1 defined analytical scalability through five dimensions; the framework’s native analytics layer (§3.2) satisfies them by consuming historized, business-keyed, feature-ready entities across the integration contract (clauses G1 and G4), so models are first-class pipeline citizens rather than external appendages (requirements R6–R9). The read-path that bypasses the serving marts is the structural expression of this answer.

RQ3 — composability. *Under what conditions can an agile relational-lineage modeling discipline (Data Vault 2.0) and a file-based Lakehouse paradigm be composed into a single coherent architecture, and what are the trade-offs?* Chapter 2 (§2.3) established that the

two are combined in practice but their interface is specified by neither paradigm. Chapter 3 specified it as the integration contract (clauses G1–G6), with a single open table format mediating relational-lineage discipline and machine-learning file access on one physical copy (invariant I7). The answer separates what was *resolved* from what was *scoped*. Resolved: the interface is specifiable, and the overlap between Satellite historization and table-version time travel is settled by making `load_dts` the system of record (assumption A3). Scoped: the join-heavy query cost is confined to the serving and feature layers, not eliminated (assumption A2); the composability claim rests partly on industry evidence; and the architecture is justified only where source heterogeneity and analytical ambition are both high (§3.5). The trade-off is therefore explicit: structural agility, native analytics, and clean composability are bought at the cost of implementation complexity and a query-performance ceiling below a dimensional warehouse’s.

Limitations

The contribution is conceptual, and its limits are stated plainly rather than concealed.

- **Conceptual, not empirical.** The framework is evaluated by criteria-based comparison, not measurement. Its ratings predict relative standing; they are not benchmarks.
- **Thin composability evidence.** The central RQ3 claim leans on vendor practice and a small peer-reviewed base, because the Data Vault 2.0–Lakehouse composition is not yet studied formally in the literature.
- **Reference model, not a system.** The framework specifies layers, a contract, and invariants; it deliberately leaves engine choice, serving latency, and tooling open (assumption A4). It is a blueprint to instantiate, not a deployed system.
- **Reasoned, not measured, performance.** The query-performance ceiling and the cost of the serving layer are argued from the paradigms’ mechanics, not from measurement.

Future Work

Each limitation points to concrete work that would extend the thesis.

- **Prototype and benchmark.** Instantiate the framework with Data Vault 2.0 on Delta or Iceberg and benchmark it across the six C2 criteria, turning the conceptual ratings into measured ones.
- **Case study at OURQUILANE.** Instantiate the framework on a real logistics and decision-support context, the natural bridge from this theoretical thesis to applied work.
- **Quantify the composability trade-off.** Study the Data Vault 2.0–Lakehouse query-performance cost empirically, closing the measurement gap identified in §2.3.

- **Extend the analytics layer.** Carry the native analytics layer from predictive toward prescriptive and optimization workloads, with feedback-driven retraining inside the governed pipeline (requirement R9).

The thesis set out to treat structural and analytical scalability as one property and to show that the two leading paradigms can be composed to serve it. Construct C3 is that composition, specified and evaluated; the work above is the path from a defensible blueprint to a measured system.

Bibliography

- AI-driven decision support system: A review integrating ML, NLP, and DL for predictive analytics. (2025). *Future Internet*, 17(9), 383. <https://doi.org/10.3390/fi17090383>
- AnalyticsCreator. (2025). *Data modeling for modern DWH: Data vault 2.0 vs kimball, inmon, anchor & mixed* [Industry source — contains BARC/Eckerson adoption figures]. Retrieved January 1, 2025, from <https://analyticscreator.ai/data-vault-2-0-vs-kimball-inmon/>
- Armbrust, M., Das, T., Sun, L., Yavuz, B., Zhu, S., Murthy, M., Torres, J., van Hovell, H., Ionescu, A., Łuszczak, A., et al. (2020). Delta lake: High-performance ACID table storage over cloud object stores. *Proceedings of the VLDB Endowment*, 13(12).
- Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021). Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. *Proceedings of CIDR 2021*.
- Bifet, A., Gavaldà, R., Holmes, G., & Pfahringer, B. (2018). *Machine learning for data streams*. MIT Press.
- Chen, H., Chiang, R. H. L., & Storey, V. C. (2012). Business intelligence and analytics: From big data to big impact. *MIS Quarterly*, 36(4), 1165–1188.
- Databricks. (2022). *Data vault best practice on the lakehouse platform* [Industry/vendor source]. Retrieved January 1, 2025, from <https://www.databricks.com/blog/2022/06/24/prescriptive-guidance-for-implementing-a-data-vault-model-on-the-databricks-lakehouse-platform.html>
- Davenport, T. H., & Harris, J. G. (2007). *Competing on analytics: The new science of winning*. Harvard Business Press.
- Feedback-driven decision support system with incremental retraining [arXiv:2508.07107]. (2025).
- Gartner. (2012). *Analytic ascendancy model* [Industry/vendor source — descriptive/diagnostic/predictive/pro maturity model]. Retrieved January 1, 2025, from <https://www.gartner.com>
- Gartner. (2018). *Gartner survey shows organizations are slow to advance in data and analytics* [Industry/vendor source — 91% of organizations sub-transformational in analytics maturity]. Retrieved January 1, 2025, from <https://www.gartner.com/en/newsroom/press-releases/2018-02-05-gartner-survey-shows-organizations-are-slow-to-advance-in-data-and-analytics>

- Giebler, C., Gröger, C., Hoos, E., Schwarz, H., & Mitschang, B. (2019). Modeling data lakes with data vault: Practical experiences, assessment, and lessons learned. *Conceptual Modeling (ER 2019)*, 11788, 63–77. https://doi.org/10.1007/978-3-030-33223-5_7
- Golfarelli, M., & Rizzi, S. (2009). *Data warehouse design: Modern principles and methodologies*. McGraw-Hill.
- Harby, A. A., & Zulkernine, F. (2024). Data lakehouse: A survey and experimental study. *Information Systems*, 127, 102460. <https://doi.org/10.1016/j.is.2024.102460>
- Inmon, W. H. (2005). *Building the data warehouse* (4th). Wiley.
- Jones, A. (2023). *Driving data quality with data contracts: A comprehensive guide to building reliable, trusted, and effective data platforms* [Industry source — originating practitioner reference for the data-contract pattern]. Packt Publishing.
- Jordan, M. I., & Mitchell, T. M. (2015). Machine learning: Trends, perspectives, and prospects. *Science*, 349(6245), 255–260.
- Kimball, R., & Caserta, J. (2004). *The data warehouse etl toolkit*. Wiley.
- Kimball, R., & Ross, M. (2013). *The data warehouse toolkit* (3rd). John Wiley & Sons.
- Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media.
- Kreps, J. (2014). *Questioning the lambda architecture* [Industry/vendor source — grey literature; originating reference for Kappa architecture]. Retrieved January 1, 2025, from <https://www.oreilly.com/radar/questioning-the-lambda-architecture/>
- Kreuzberger, D., Köhl, N., & Hirschl, S. (2023). Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 11, 31866–31879. <https://doi.org/10.1109/ACCESS.2023.3262138>
- Linstedt, D., & Olschimke, M. (2015). *Building a scalable data warehouse with data vault 2.0*. Morgan Kaufmann.
- Marz, N., & Warren, J. (2015). *Big data: Principles and best practices of scalable real-time data systems* [Industry/vendor source — grey literature; originating reference for Lambda architecture]. Manning Publications.
- Moermans, K., & Olschimke, M. (2023). *Data vault 2.0 using databricks lakehouse architecture on azure* [Industry/vendor source]. Retrieved January 1, 2025, from <https://techcommunity.microsoft.com/t5/azure-synapse-analytics-blog/data-vault-2-0-using-databricks-lakehouse-architecture-on-azure/ba-p/3689781>
- Moronta, S. (2026). *Data vault 2.0 in the lakehouse era: Advanced architecture and patterns* [Industry source — Towards Data Engineering (Medium)]. Retrieved January 1, 2026, from <https://towardsdatascience.com>
- Nambiar, A., & Mundra, D. (2022). An overview of data warehouse and data lake in modern enterprise data management. *Big Data and Cognitive Computing*, 6(4), 132. <https://doi.org/10.3390/bdcc6040132>

- Schneider, J., et al. (2024). The lakehouse: State of the art on concepts and technologies. *SN Computer Science*, 5.
- Understanding data and analytics maturity: A systematic review of maturity model composition. (2024). *Schmalenbach Journal of Business Research*. <https://doi.org/10.1007/s41471-024-00205-2>
- Varga, M. (2014). *Data vault standards, naming conventions and modeling principles* (tech. rep.). Data Vault Alliance.
- Vassiliadis, P. (2009). A survey of extract-transform-load technology. *International Journal of Data Warehousing and Mining*, 5(3), 1–27.
- Watson, H. J. (2009). Tutorial: Business Intelligence — past, present, and future. *Communications of the Association for Information Systems*, 25(39), 487–510. <https://doi.org/10.17705/1CAIS.02539>

Appendix A

Annexe A — [TODO: Title]

Appendix B

Annexe B — [TODO: Title]